

Demystifying NVSHMEM: A System-Level Analysis on Symmetric Memory and Device-Initiated Operations in GPU Communication

Yijun Ma^{1*}, Siyuan Shen^{1*}, Tiancheng Chen¹, Akhil Langer², Jiri Kraus², Benjamin Glick², Craig Belusar², Jeff Hammond², Torsten Hoefler¹

¹ ETH Zurich, ² NVIDIA Corporation



Background

Background

Petascale → Exascale Computing ↑



Background

Petascale → Exascale Computing ↑



Communication Bottleneck

- Communication did not scale at the same rate as compute

Background

Petascale → Exascale Computing ↑



Communication Bottleneck

- Communication did not scale at the same rate as compute
- CPU-driven communication is the bottleneck



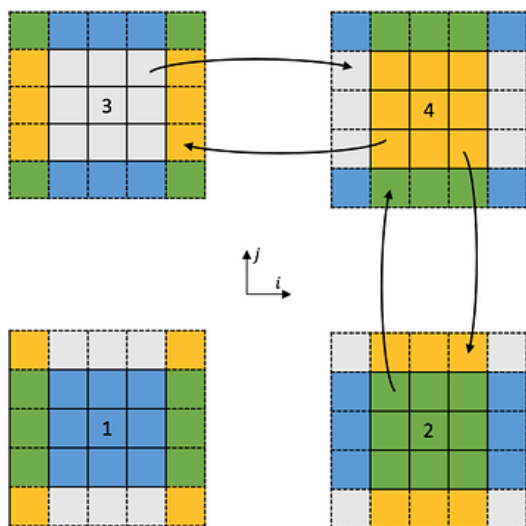
Background

Background

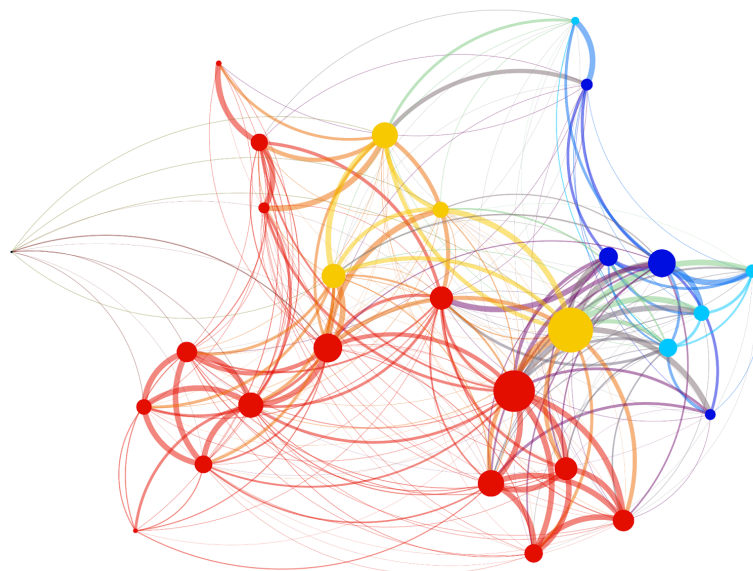
- CPU-driven communication is the bottleneck for ...
 - fine-grain point-to-point communication patterns
 - data-dependent scenarios / restricted to subsets of threads

Background

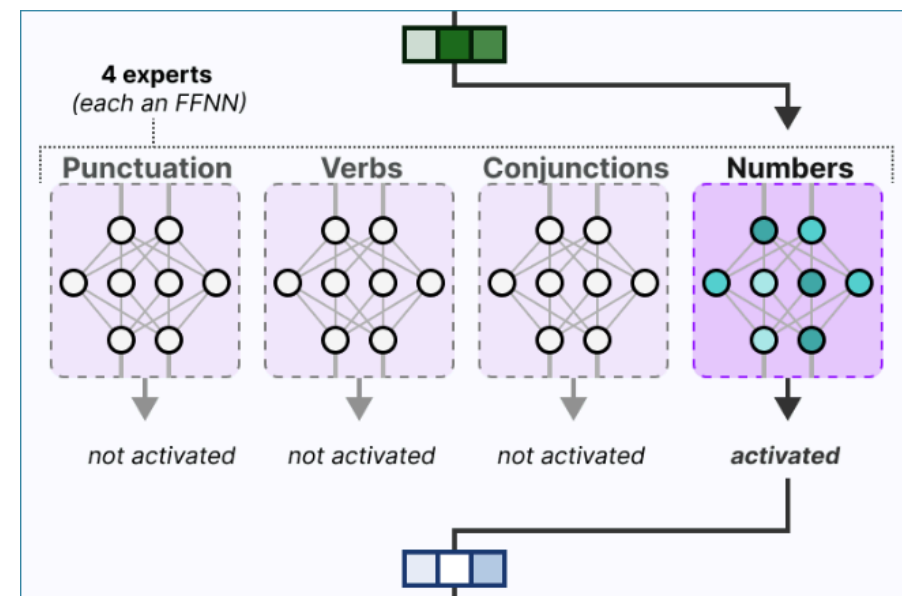
- CPU-driven communication is the bottleneck for ...
 - fine-grain point-to-point communication patterns
 - data-dependent scenarios / restricted to subsets of threads



stencil halo exchange

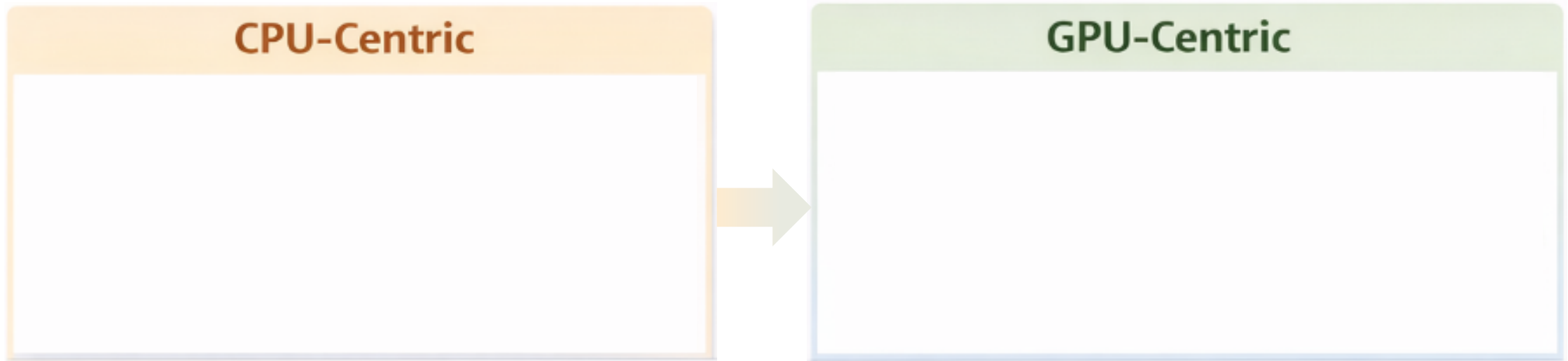


irregular graph

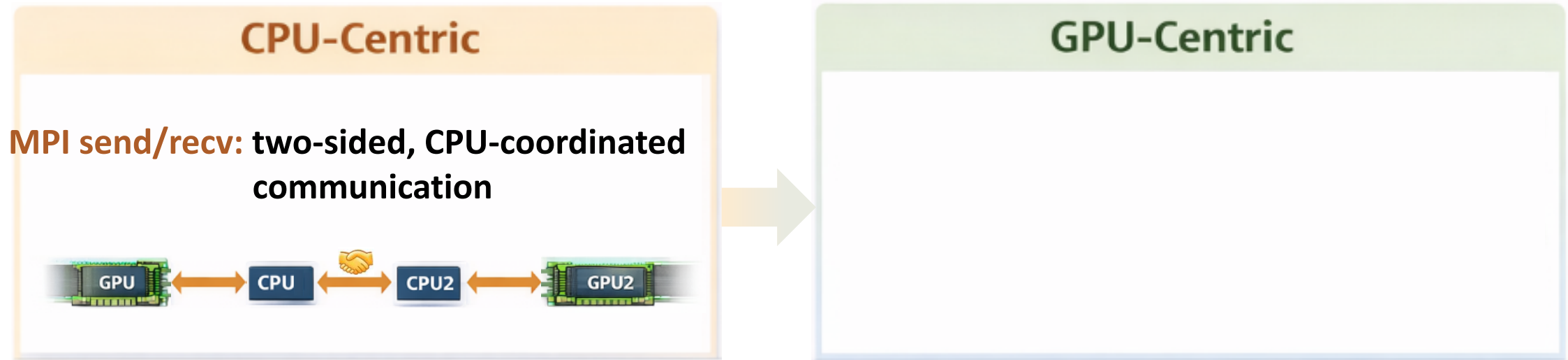


sparse expert models

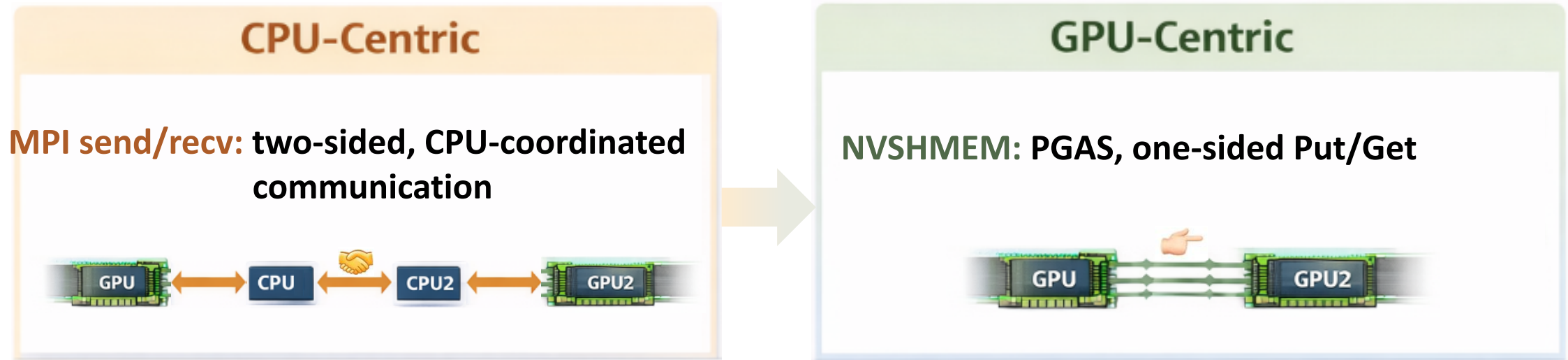
Background: From CPU-Centric to GPU-Centric Communication



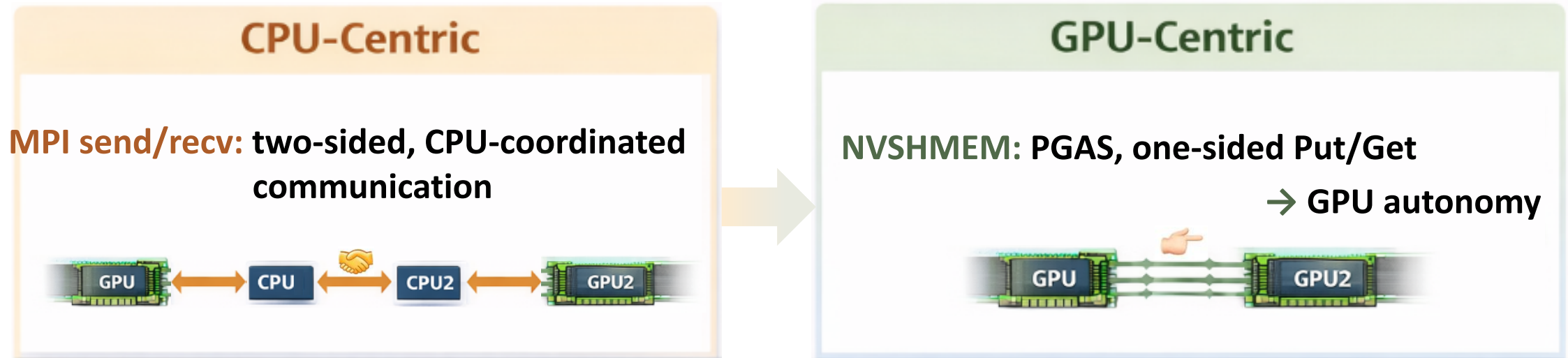
Background: From CPU-Centric to GPU-Centric Communication



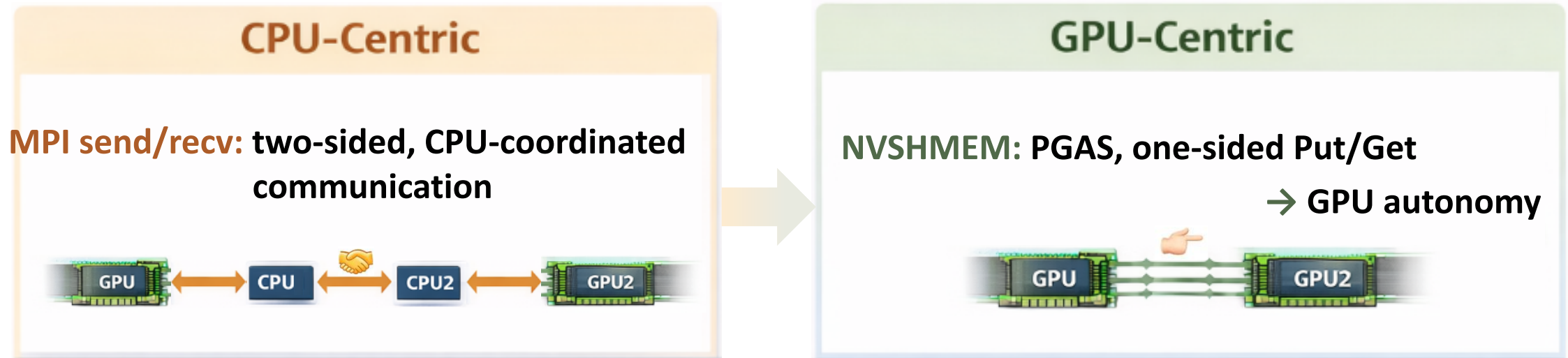
Background: From CPU-Centric to GPU-Centric Communication



Background: From CPU-Centric to GPU-Centric Communication



Background: From CPU-Centric to GPU-Centric Communication



- Decouples synchronization and data
- Device-initiated communication

The GPU Communication Landscape

Library	Communication Style	Where communication is initiated
---------	---------------------	----------------------------------

CUDA-aware MPI and UPC++ / GASNet-EX can move GPU-resident data, but the control path stays on the host.

The GPU Communication Landscape

Library	Communication Style	Where communication is initiated
CUDA-aware MPI	MPI model, two-sided, rank-based	Host
UPC++ / GASNet-EX	Portable PGAS, one-sided	Host control path

CUDA-aware MPI and UPC++ / GASNet-EX can move GPU-resident data, but the control path stays on the host.

The GPU Communication Landscape

Library	Communication Style	Where communication is initiated
CUDA-aware MPI	MPI model, two-sided, rank-based	Host
UPC++ / GASNet-EX	Portable PGAS, one-sided	Host control path
NCCL (conventional)	Dense, regular collectives	Host
NCCL Device API / GIN	Device-initiated operations, collective-centric	Device

CUDA-aware MPI and UPC++ / GASNet-EX can move GPU-resident data, but the control path stays on the host.

The GPU Communication Landscape

Library	Communication Style	Where communication is initiated
CUDA-aware MPI	MPI model, two-sided, rank-based	Host
UPC++ / GASNet-EX	Portable PGAS, one-sided	Host control path
NCCL (conventional)	Dense, regular collectives	Host
NCCL Device API / GIN	Device-initiated operations, collective-centric	Device
rocSHMEM (ROCm)	OpenSHMEM model	Device
Intel SHMEM (oneAPI / SYCL)	OpenSHMEM model	Device

CUDA-aware MPI and UPC++ / GASNet-EX can move GPU-resident data, but the control path stays on the host.

The GPU Communication Landscape

Library	Communication Style	Where communication is initiated
CUDA-aware MPI	MPI model, two-sided, rank-based	Host
UPC++ / GASNet-EX	Portable PGAS, one-sided	Host control path
NCCL (conventional)	Dense, regular collectives	Host
NCCL Device API / GIN	Device-initiated operations, collective-centric	Device
GICC (Slingshot)	Triggered operations	Device
rocSHMEM (ROCm)	OpenSHMEM model	Device
Intel SHMEM (oneAPI / SYCL)	OpenSHMEM model	Device

CUDA-aware MPI and UPC++ / GASNet-EX can move GPU-resident data, but the control path stays on the host.

The GPU Communication Landscape

Library	Communication Style	Where communication is initiated
CUDA-aware MPI	MPI model, two-sided, rank-based	Host
UPC++ / GASNet-EX	Portable PGAS, one-sided	Host control path
NCCL (conventional)	Dense, regular collectives	Host
NCCL Device API / GIN	Device-initiated operations, collective-centric	Device
GICC (Slingshot)	Triggered operations	Device
rocSHMEM (ROCm)	OpenSHMEM model	Device
Intel SHMEM (oneAPI / SYCL)	OpenSHMEM model	Device
NVSHMEM	OpenSHMEM model, PGAS, fine-grained one-sided	Device (thread / warp / block)

- Device-initiated PGAS communication is a **broader vendor trend**, not a feature unique to NVIDIA

NVSHMEM Concepts

NVSHMEM Concepts

- A runtime that implements the OpenSHMEM standard specifically optimized for NVIDIA GPU clusters

NVSHMEM Concepts

- A runtime that implements the OpenSHMEM standard specifically optimized for NVIDIA GPU clusters
- **Device-side APIs**
 - Overlap communication with computation, hiding latency
 - Fine-grained parallelism (thread/warp/block level) → perfect for customized kernel fusion

NVSHMEM Concepts

- A runtime that implements the OpenSHMEM standard specifically optimized for NVIDIA GPU clusters
- **Device-side APIs**
 - Overlap communication with computation, hiding latency
 - Fine-grained parallelism (thread/warp/block level) → perfect for customized kernel fusion

- Library setup/exit
- Memory management
- Team management
- Collective launch

NVSHMEM Concepts

- A runtime that implements the OpenSHMEM standard specifically optimized for NVIDIA GPU clusters
- **Device-side APIs**
 - Overlap communication with computation, hiding latency
 - Fine-grained parallelism (thread/warp/block level) → perfect for customized kernel fusion

Host-only APIs

- Library setup/exit
- Memory management
- Team management
- Collective launch

NVSHMEM Concepts

- A runtime that implements the OpenSHMEM standard specifically optimized for NVIDIA GPU clusters
- **Device-side APIs**
 - Overlap communication with computation, hiding latency
 - Fine-grained parallelism (thread/warp/block level) → perfect for customized kernel fusion

Host-only APIs

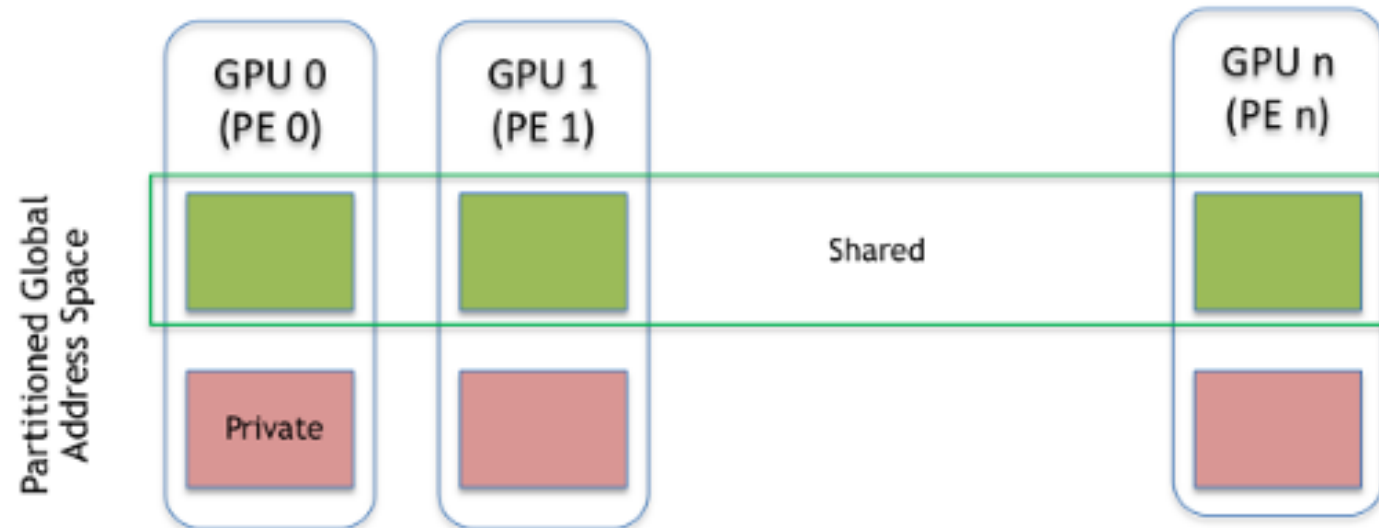
- Library setup/exit
- Memory management
- Team management
- Collective launch

Dual interface (host/device)

- Remote memory access
- Atomic memory operations
- Collective communication
- Signaling operations
- Point-to-point synchronization
- Memory ordering

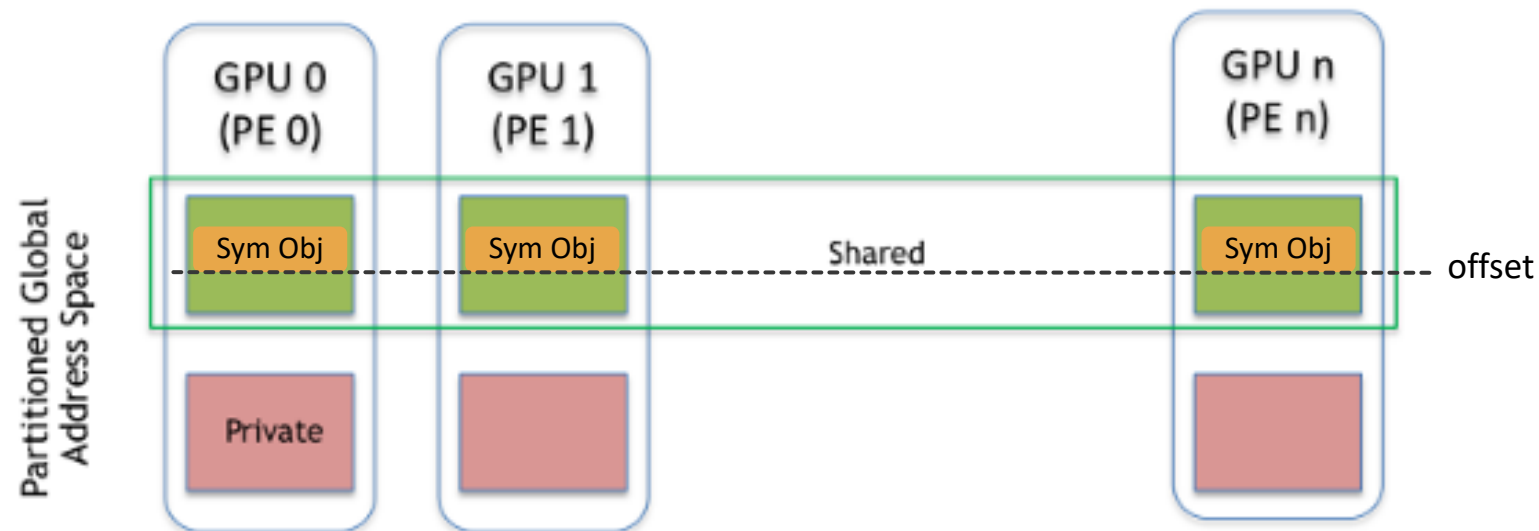
NVSHMEM Concepts

- A runtime that implements the OpenSHMEM standard specifically optimized for NVIDIA GPU clusters (Device-side APIs)
- A SHMEM library for the *partitioned global address space (PGAS)* parallel programming model



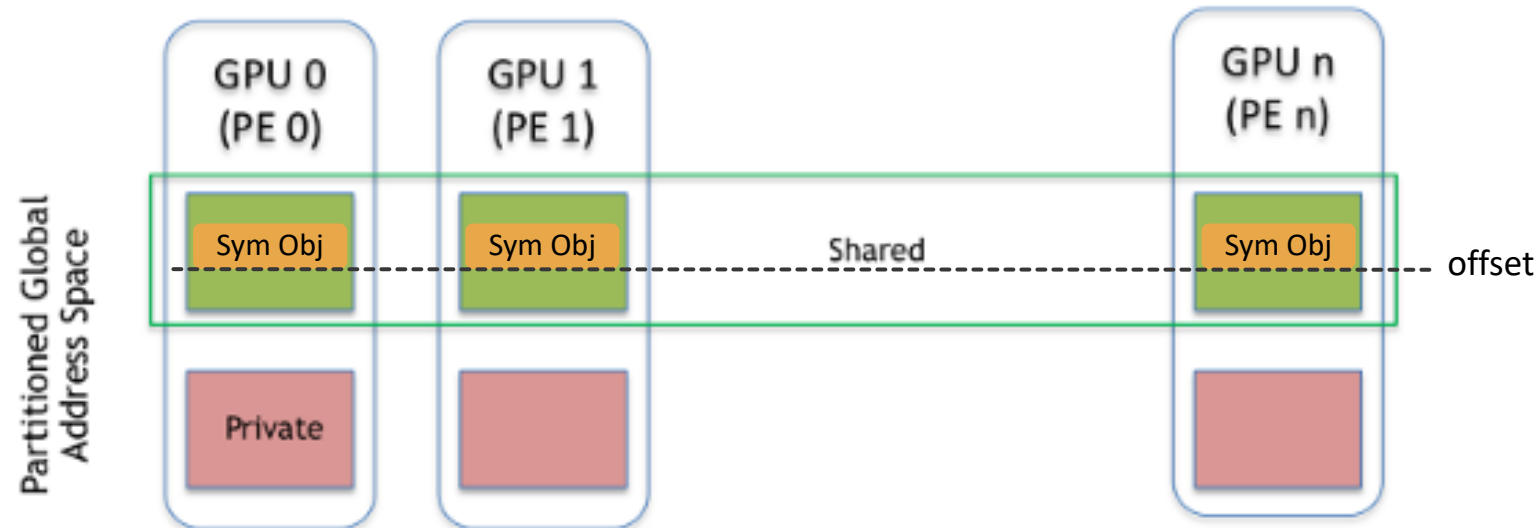
NVSHMEM Concepts

- A runtime that implements the OpenSHMEM standard specifically optimized for NVIDIA GPU clusters (Device-side APIs)
- A SHMEM library for the *partitioned global address space (PGAS)* parallel programming model
- **Symmetric Memory**
 - A symmetric object exists at the same relative virtual offset across every processing element (PE)



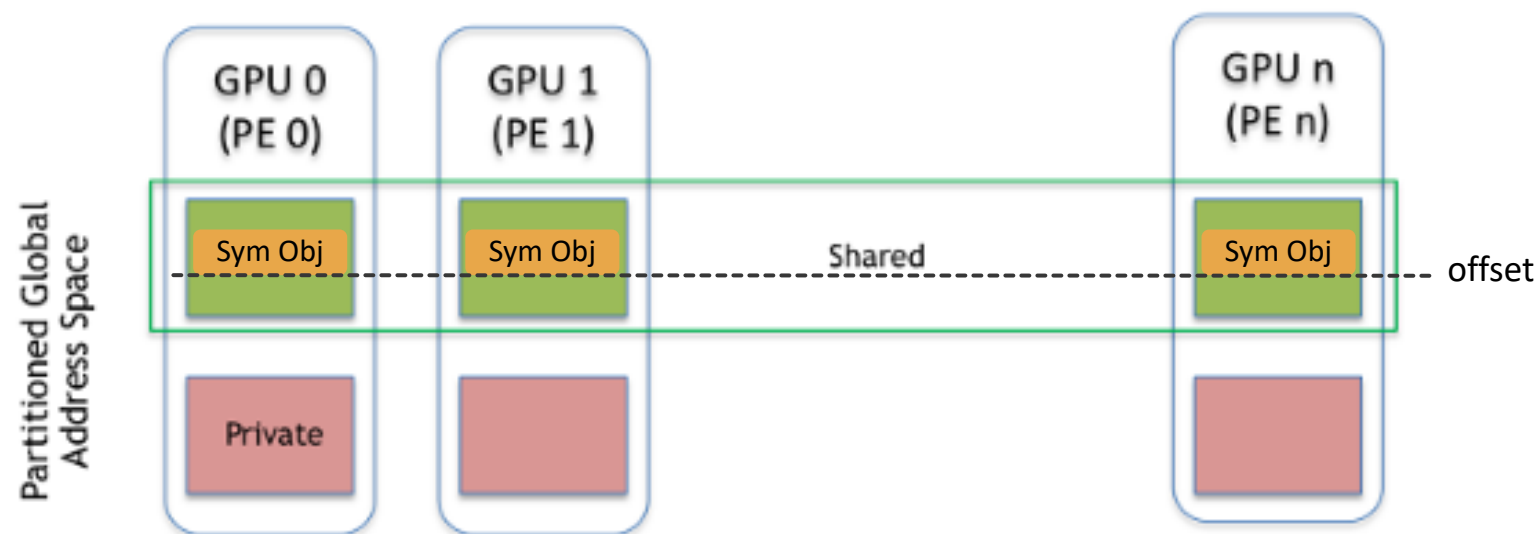
NVSHMEM Concepts

- A runtime that implements the OpenSHMEM standard specifically optimized for NVIDIA GPU clusters (Device-side APIs)
- A SHMEM library for the *partitioned global address space (PGAS)* parallel programming model
- **Symmetric Memory**
 - A symmetric object exists at the same relative virtual offset across every processing element (PE)
(enables **direct remote addressing**)



NVSHMEM Concepts

- A runtime that implements the OpenSHMEM standard specifically optimized for NVIDIA GPU clusters (Device-side APIs)
- A SHMEM library for the *partitioned global address space (PGAS)* parallel programming model
- **Symmetric Memory**
 - A symmetric object exists at the same relative virtual offset across every processing element (PE) (enables **direct remote addressing**) and has a corresponding memory allocation on every PE



NVSHMEM Concepts

- A runtime that implements the OpenSHMEM standard specifically optimized for NVIDIA GPU clusters (Device-side APIs)
- A SHMEM library for the *partitioned global address space (PGAS)* parallel programming model
- **Symmetric Memory**
 - A symmetric object exists at the same relative virtual offset across every processing element (PE) (enables **direct remote addressing**) and has a corresponding memory allocation on every PE
- **Relaxed Ordering**
 - No implicit ordering (No $W \rightarrow W$, No $R \rightarrow R$)

Symmetric Memory Addressing

Symmetric Memory Addressing

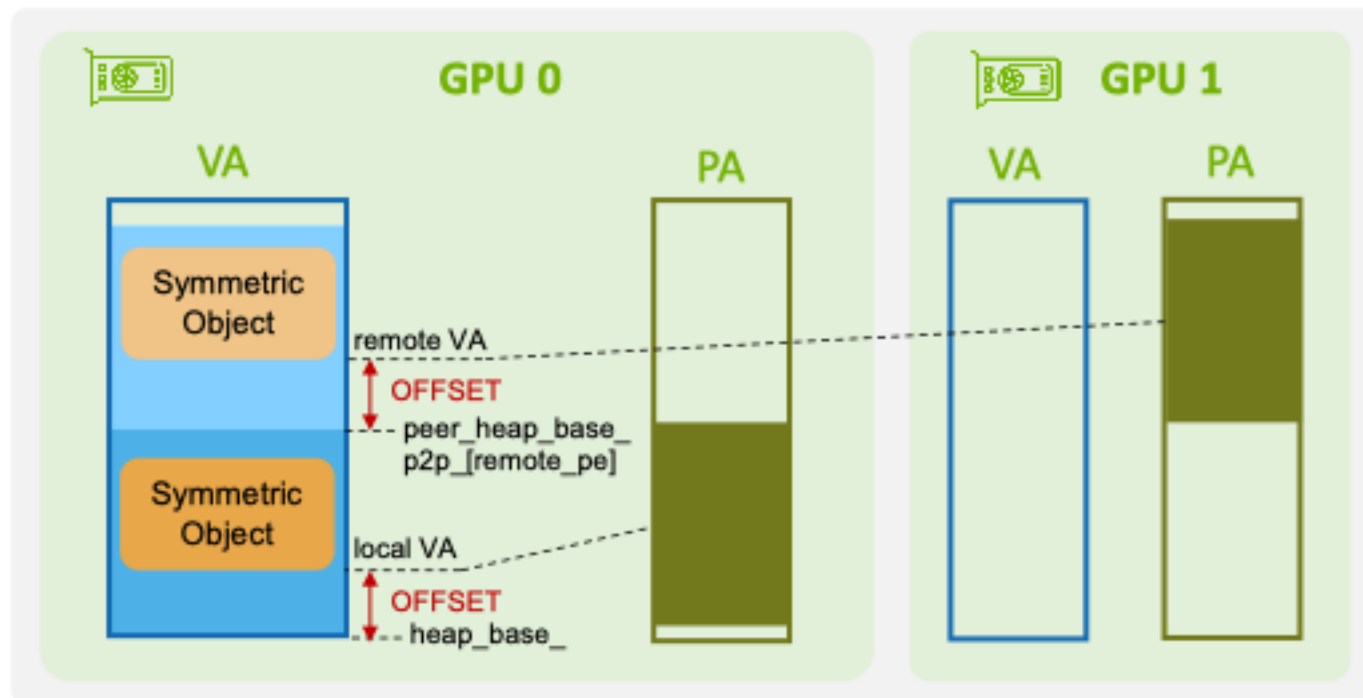
- A symmetric object keeps the **same offset** inside every PE's heap

Symmetric Memory Addressing

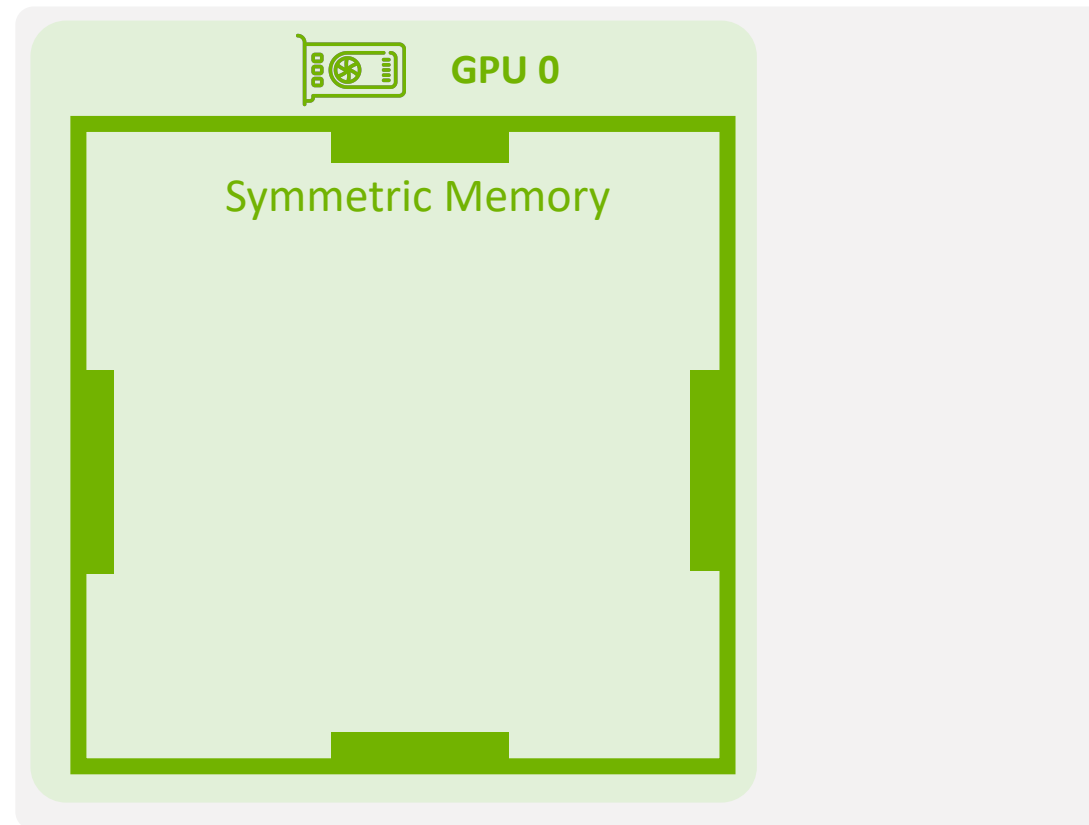
- A symmetric object keeps the **same offset** inside every PE's heap

$$\text{dest_remote} = \text{peer_heap_base_}\{\text{p2p_}|\text{remote_}\}[\text{remote_pe}] + (\text{dest_local} - \text{heap_base_})$$

fast/slow path

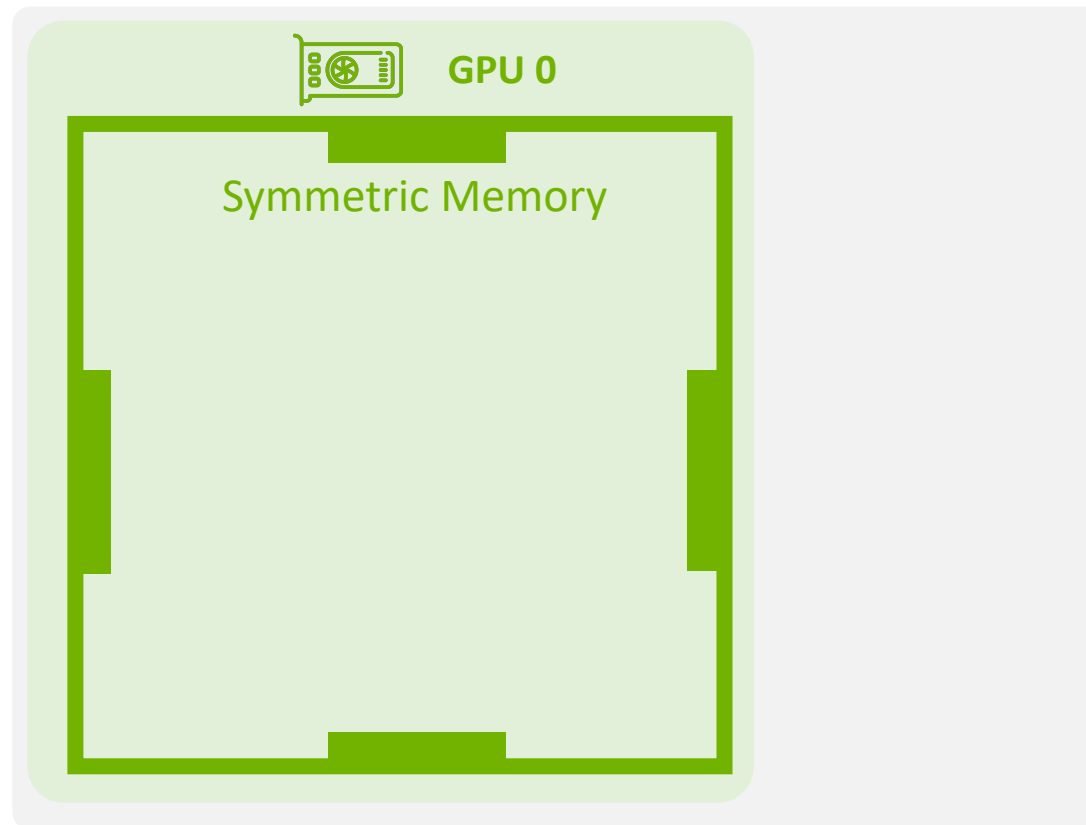


Symmetric Memory Addressing



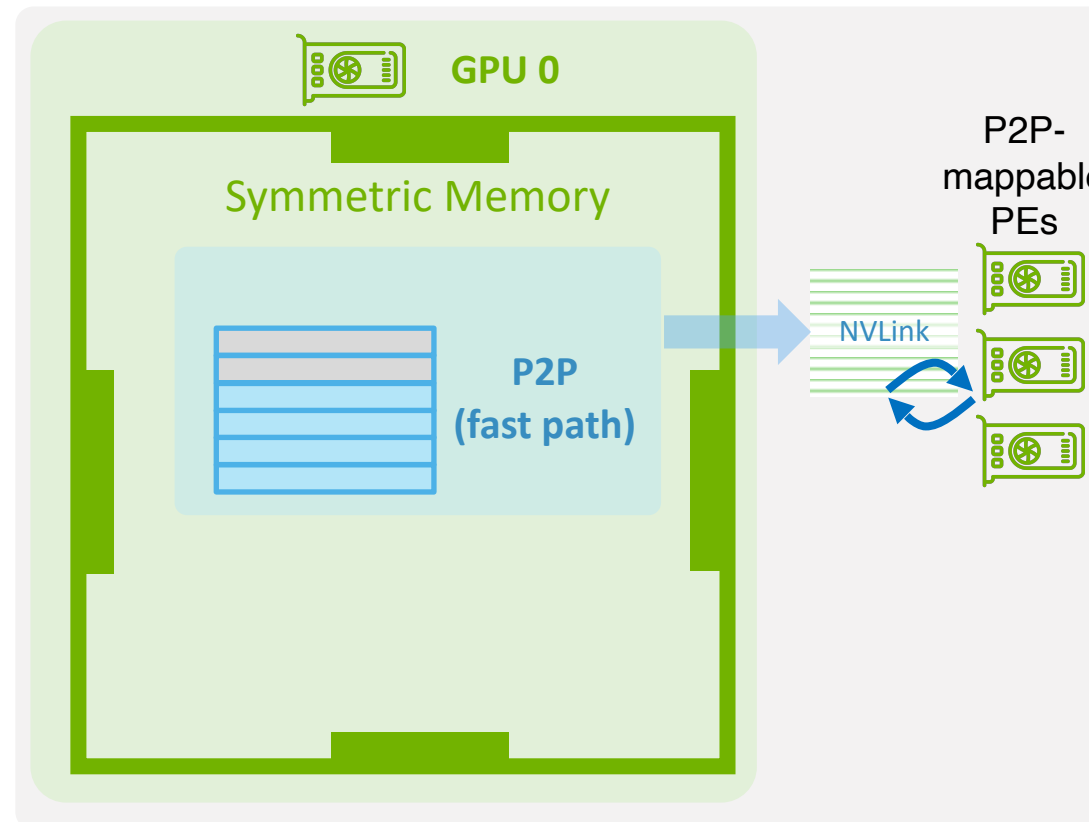
Symmetric Memory Addressing

- Symmetric object addressed by `<symmetric_address, destination_PE>`



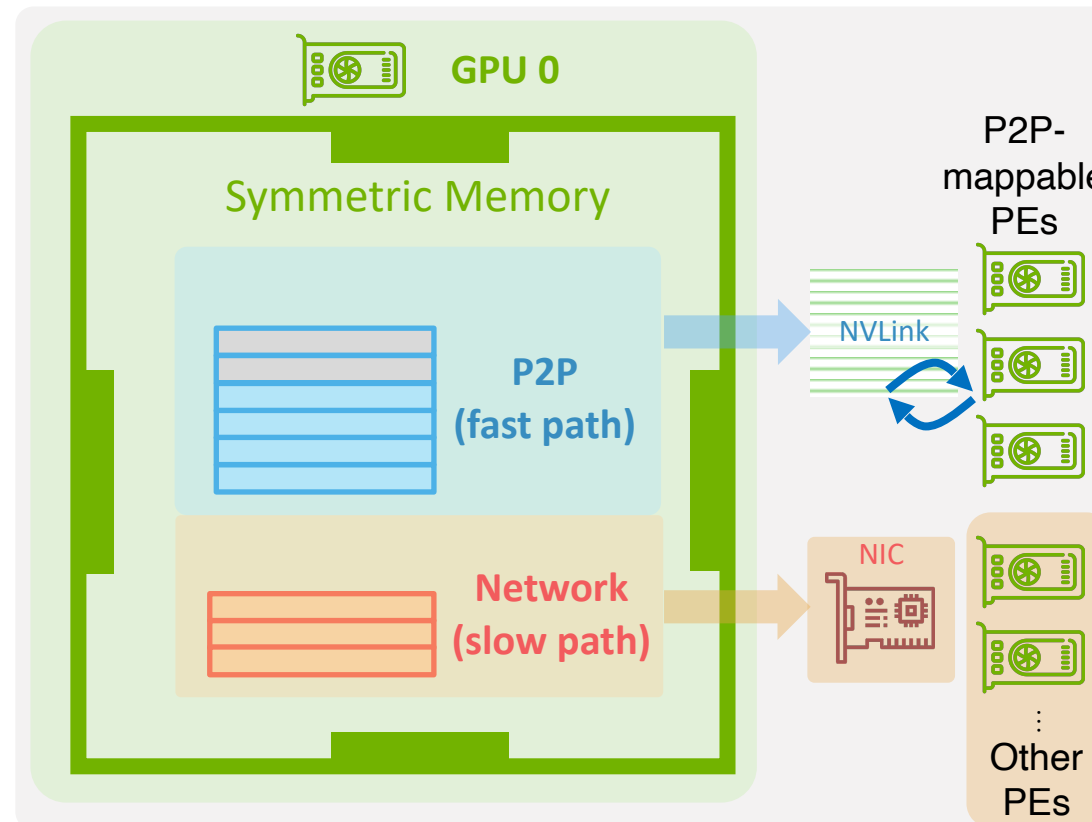
Symmetric Memory Addressing

- Symmetric object addressed by `<symmetric_address, destination_PE>`
 - If destination PE is **P2P direct-mapped**: **fast path** via the map transport, e.g. NVLink

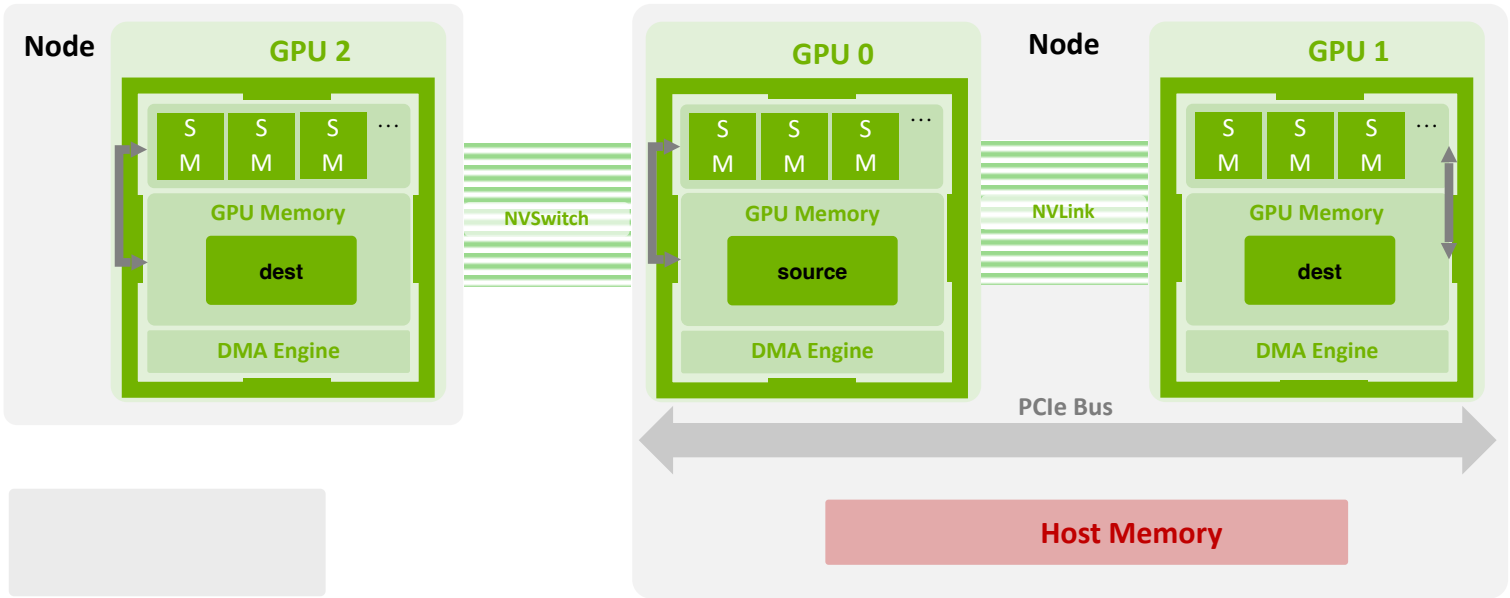


Symmetric Memory Addressing

- Symmetric object addressed by `<symmetric_address, destination_PE>`
 - If destination PE is **P2P direct-mapped: fast path** via the map transport, e.g. NVLink
 - Otherwise: **slow path via network**, e.g. InfiniBand
 - Everything is GPU-initiated

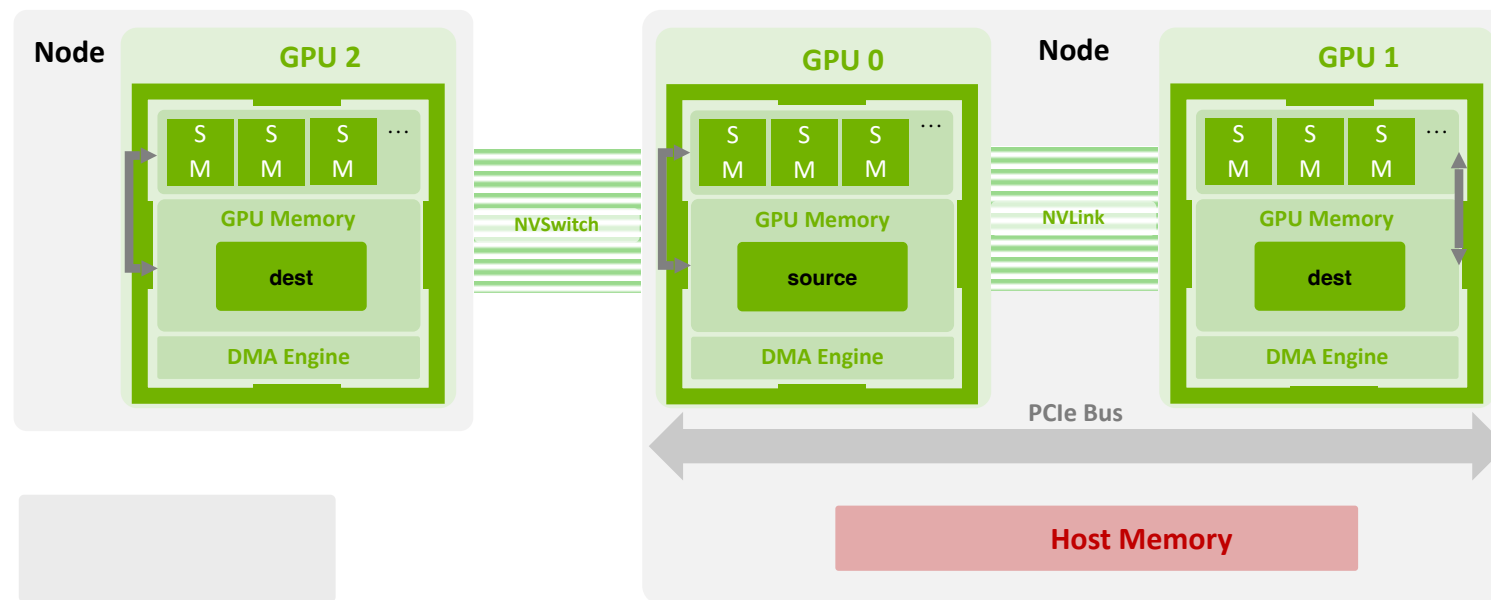


Remote Memory Access – Fast Path



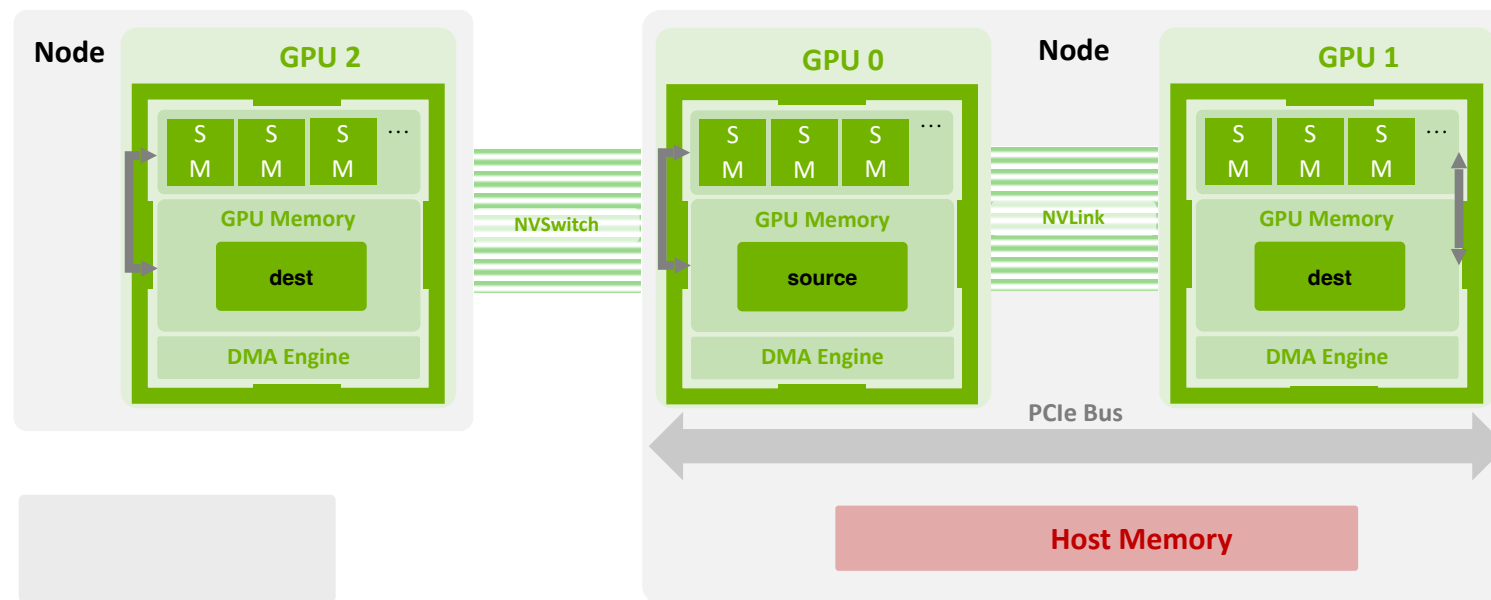
Remote Memory Access – Fast Path

- **Device-side:** the mapped peer heap turns RMA into **SM-issued loads and stores** inside the kernel



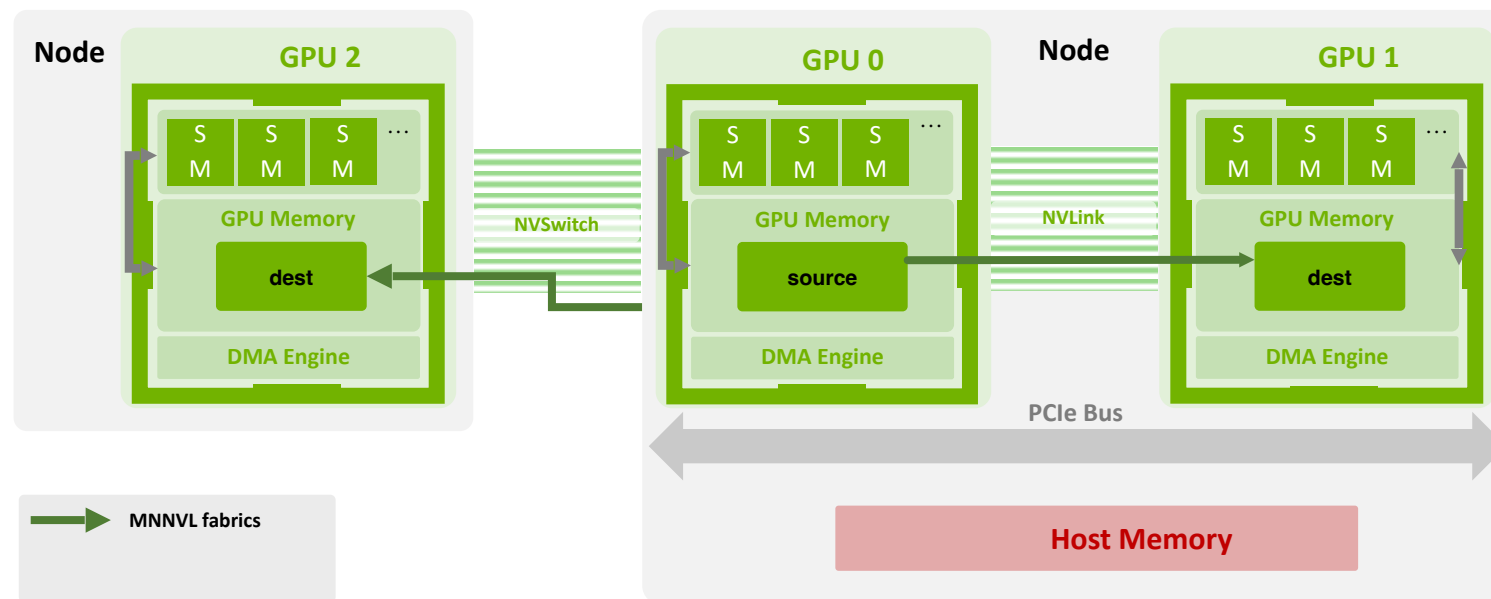
Remote Memory Access – Fast Path

- **Device-side:** the mapped peer heap turns RMA into **SM-issued loads and stores** inside the kernel
- **Host-side:** NVSHMEM issues CUDA copies instead, which may use asynchronous **copy engines**



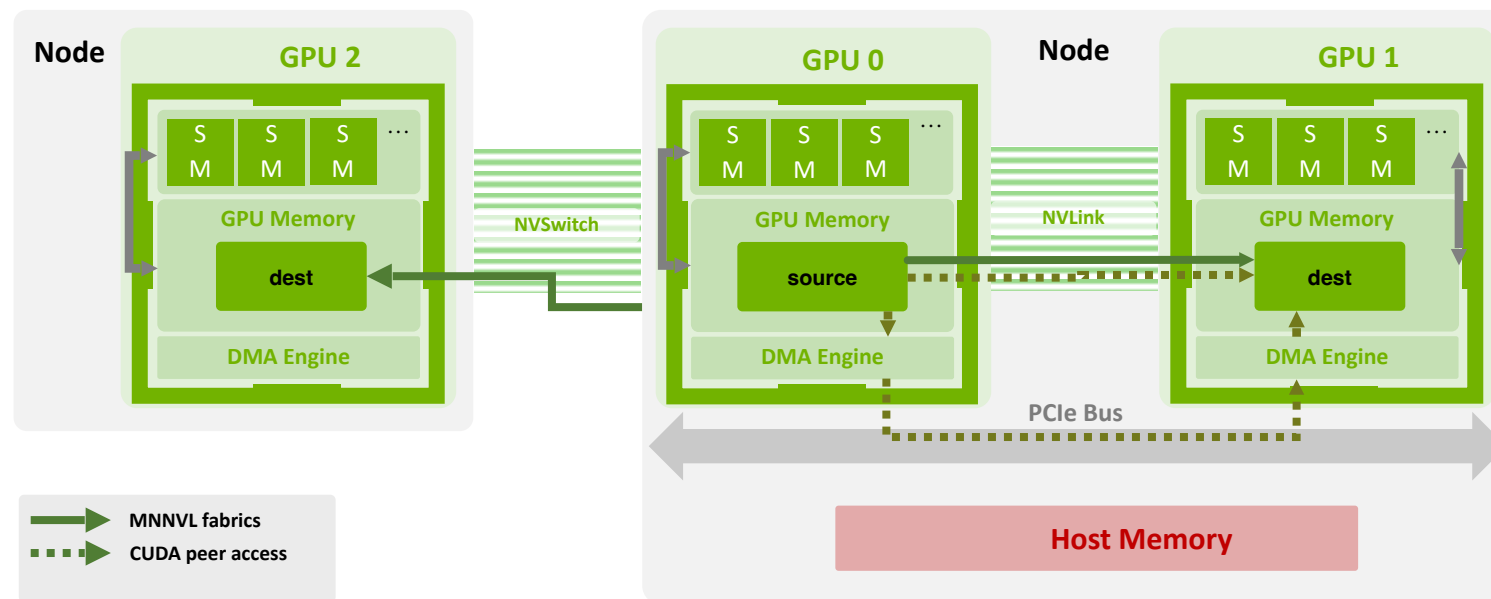
Remote Memory Access – Fast Path

- **Device-side:** the mapped peer heap turns RMA into **SM-issued loads and stores** inside the kernel
- **Host-side:** NVSHMEM issues CUDA copies instead, which may use asynchronous **copy engines**



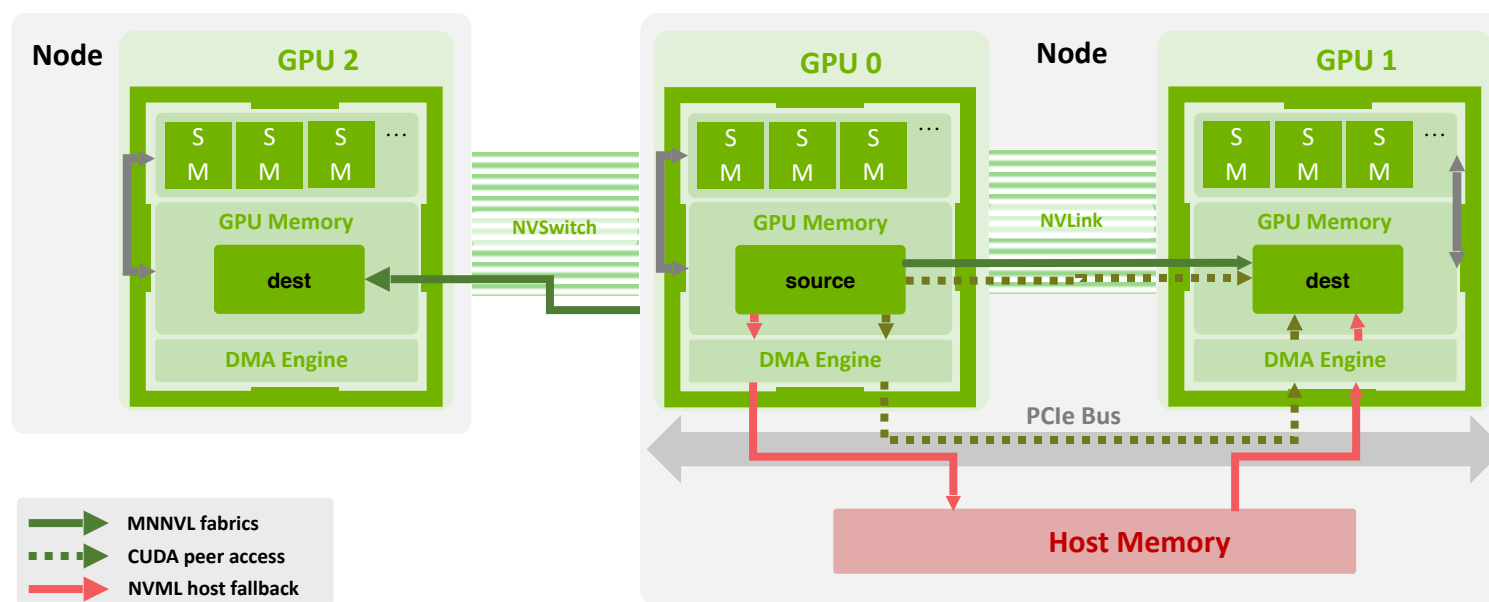
Remote Memory Access – Fast Path

- **Device-side:** the mapped peer heap turns RMA into **SM-issued loads and stores** inside the kernel
- **Host-side:** NVSHMEM issues CUDA copies instead, which may use asynchronous **copy engines**



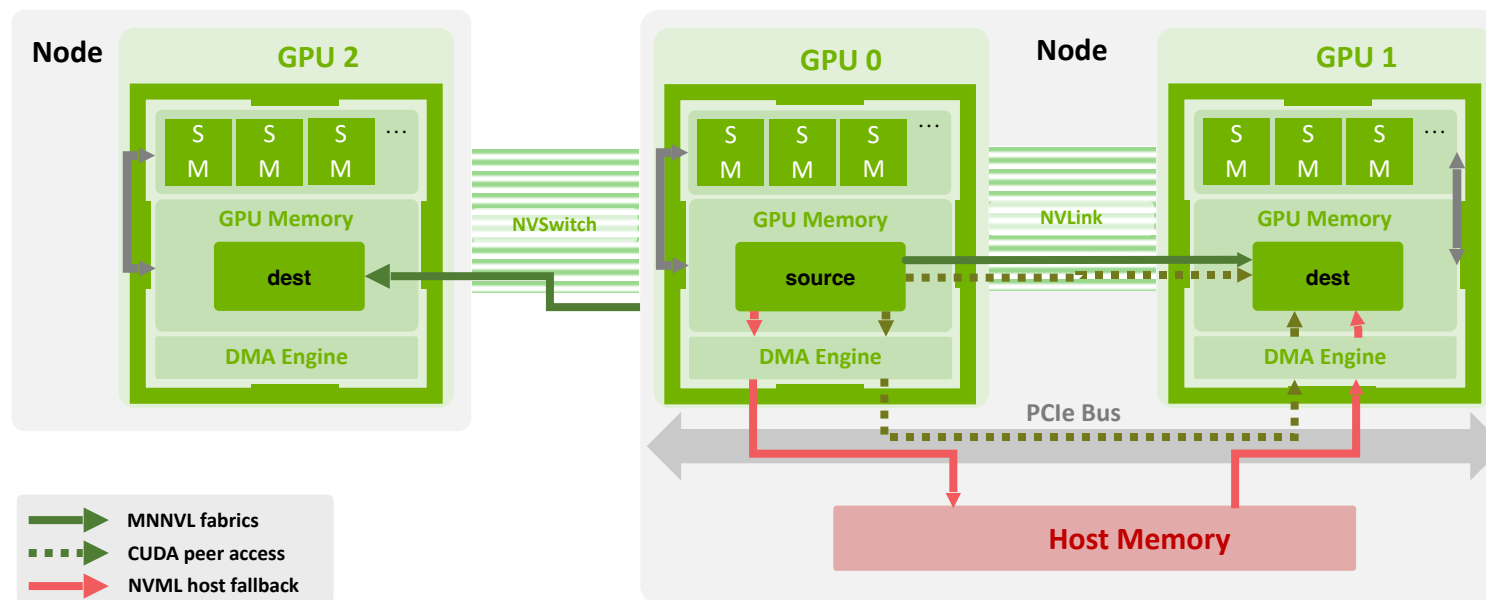
Remote Memory Access – Fast Path

- **Device-side:** the mapped peer heap turns RMA into **SM-issued loads and stores** inside the kernel
- **Host-side:** NVSHMEM issues CUDA copies instead, which may use asynchronous **copy engines**

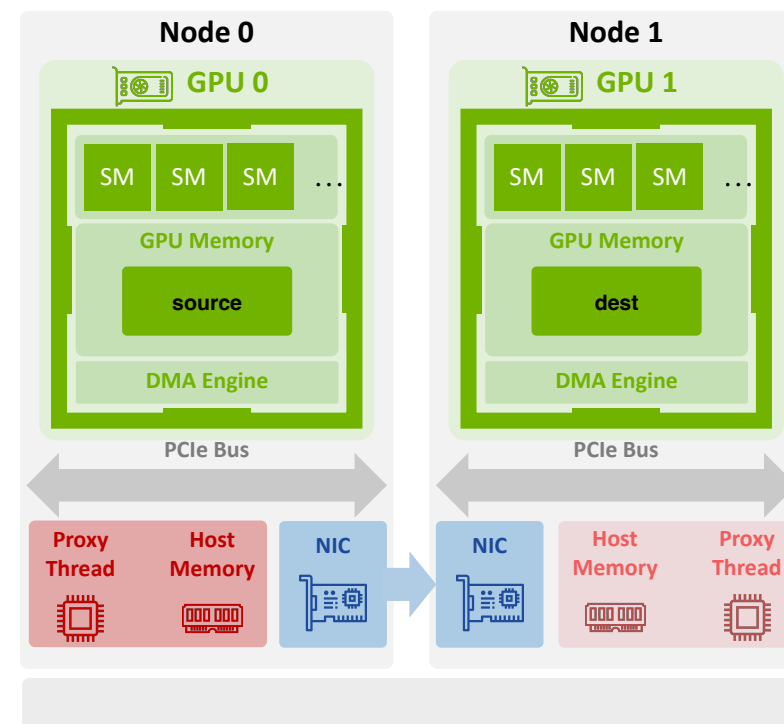


Remote Memory Access – Fast Path

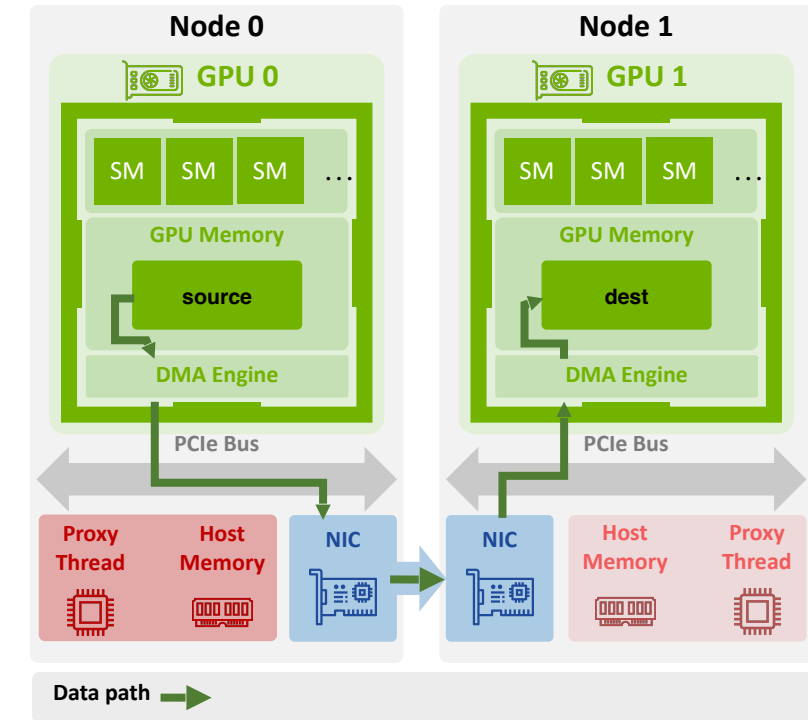
- **Device-side:** the mapped peer heap turns RMA into **SM-issued loads and stores** inside the kernel
- **Host-side:** NVSHMEM issues CUDA copies instead, which may use asynchronous **copy engines**
- Mapping established via **MNNVL fabrics** (NVLink, NVSwitch) · **CUDA peer access** (intranode NVLink, PCIe) · **host-memory fallback path**



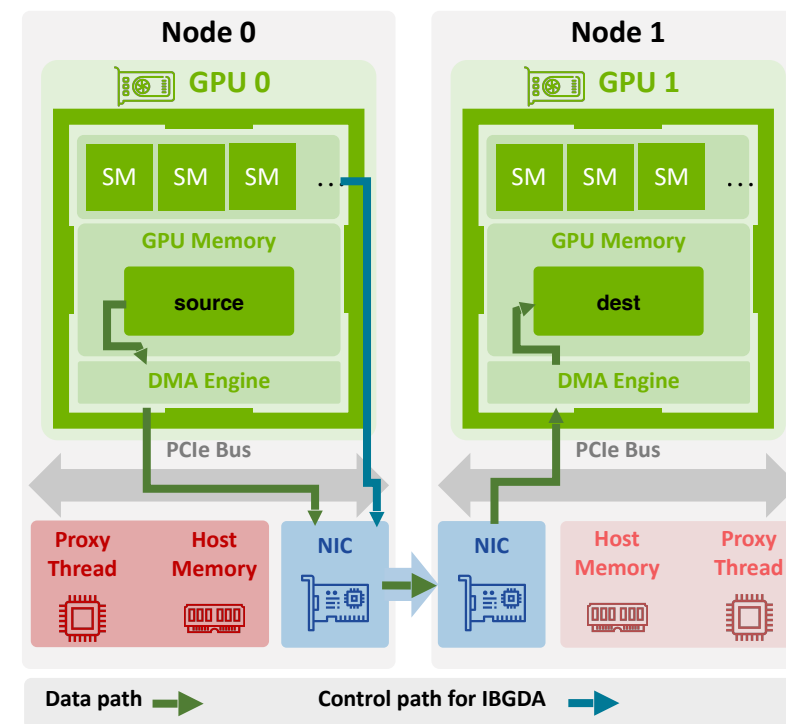
Remote Memory Access – Slow Path



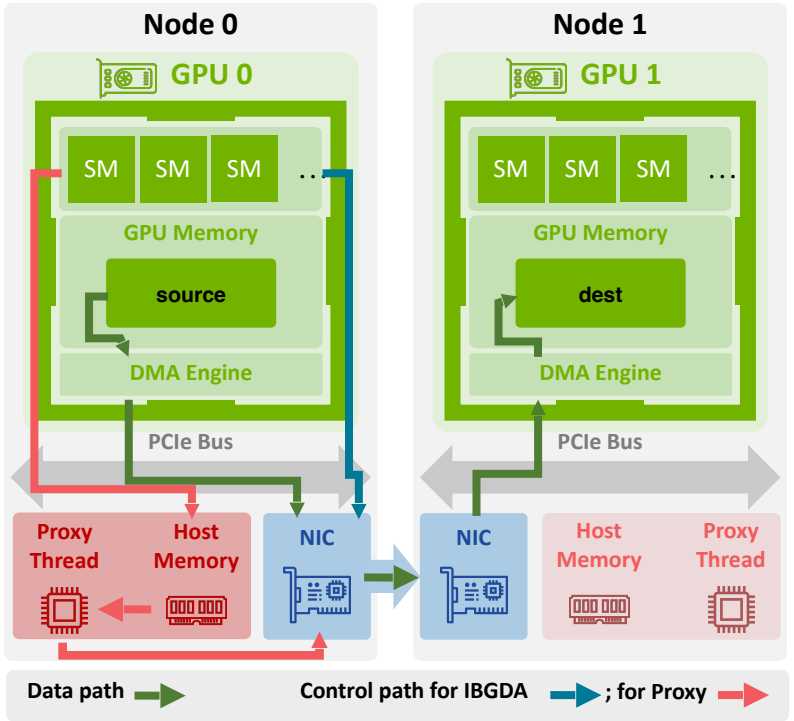
Remote Memory Access – Slow Path



Remote Memory Access – Slow Path

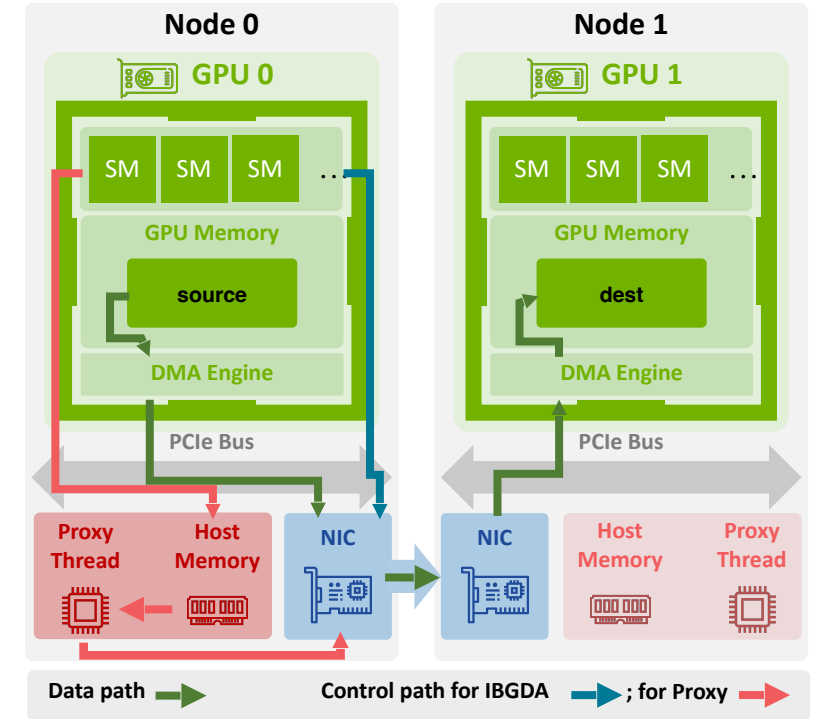


Remote Memory Access – Slow Path



Remote Memory Access – Slow Path

- Data path is zero-copy (GPUDirect RDMA)
- Supports InfiniBand GPUDirect Async (IBGDA)
 - Control path does not involve CPU
 - SM directly controls NIC
- Recent versions expose QP-specific APIs over IB QP selection



NVSHMEM Teams

NVSHMEM Teams

- Holds almost **no runtime state**; a lightweight handle identifying a PE group, which may encode topology information

NVSHMEM Teams

- Holds almost **no runtime state**; a lightweight handle identifying a PE group, which may encode topology information
- Each team owns **exactly one set of internal resources** (pSync buffer, etc.), so it cannot be safely used by concurrent collective invocations from the same PE
 - forces collectives onto a **single CTA**

Collective Algorithms

Collective Algorithms

Collective

Broadcast

AlltoAll

FCollect
(AllGather)

Reduce
(AllReduce)

ReduceScatter

Collective Algorithms

- Hardware- and topology-aware algorithms possibly utilizing low-latency (LL) data formats

Collective	Algorithm
Broadcast	Bruteforce put-to-all
	k -ary flat tree
	Topology-aware hierarchical tree
AlltoAll	P2P / general all-push
FCollect (AllGather)	NVLS one-shot Generic all-push
Reduce (AllReduce)	NVLS one-shot
	NVLS two-shot
	k -ary recursive exchange
	Hierarchical fcollect
	(Segmented) linear AllReduce
ReduceScatter	NVLS one-shot Generic all-push

Collective Algorithms

- Hardware- and topology-aware algorithms possibly utilizing low-latency (LL) data formats
- Selected at runtime from a **rule-based decision tree** (NVLS availability, P2P connectivity, datatype...)

Collective	Algorithm
Broadcast	Bruteforce put-to-all
	k -ary flat tree
	Topology-aware hierarchical tree
AlltoAll	P2P / general all-push
FCollect (AllGather)	NVLS one-shot Generic all-push
Reduce (AllReduce)	NVLS one-shot
	NVLS two-shot
	k -ary recursive exchange
	Hierarchical fcollect
	(Segmented) linear AllReduce
ReduceScatter	NVLS one-shot Generic all-push

Collective Algorithms

- Hardware- and topology-aware algorithms possibly utilizing low-latency (LL) data formats
- Selected at runtime from a **rule-based decision tree** (NVLS availability, P2P connectivity, datatype...)

M IS THE MESSAGE SIZE, N IS THE NUMBER OF PES

Collective	Algorithm	LL / LL128	Communication Volume	Latency, # Synchronizations
Broadcast	Bruteforce put-to-all	No	$O(MN)$	$O(1)$
	k -ary flat tree	LL only	$O(MN)$	$O(\log_k N)$
	Topology-aware hierarchical tree	LL only	$O(MN)$	$O(\log_k N_{\text{remote}} + \log_k N_{\text{intra}})$
AlltoAll	P2P / general all-push	No	$O(MN^2)$	$O(1)$
FCollect (AllGather)	NVLS one-shot	LL only	$O(MN^2)$	$O(1)$
	Generic all-push	Both	$O(MN^2)$	$O(1)$
Reduce (AllReduce)	NVLS one-shot	No	$O(MN^2)$	$O(1)$
	NVLS two-shot	No	$O(MN)$	$O(1)$
	k -ary recursive exchange	No	$O(MN \times k \log_k N)$	$O(\log_k N)$
	Hierarchical fcollect	Inherited from FCollect	$O(MN^2)$	$O(1)$
	(Segmented) linear AllReduce	No	$O(MN^2)$	Direct LD/ST: $O(1)$; Segmented: $O(SN)$
ReduceScatter	NVLS one-shot	No	$O(MN^2)$	$O(1)$
	Generic all-push	No	$O(MN^2)$	$O(1)$

AND $S = \lceil M/B_{\text{seg}} \rceil$ DENOTES THE NUMBER OF SCRATCH-LIMITED SEGMENTS IN SEGMENTED ALLREDUCE.

Collective Algorithms

- Hardware- and topology-aware algorithms possibly utilizing low-latency (LL) data formats
- Selected at runtime from a **rule-based decision tree** (NVLS availability, P2P connectivity, datatype...)

M IS THE MESSAGE SIZE, N IS THE NUMBER OF PEs

Collective	Algorithm	LL / LL128	Communication Volume	Latency, # Synchronizations
Broadcast	Bruteforce put-to-all	No	$O(MN)$	$O(1)$
	k -ary flat tree	LL only	$O(MN)$	$O(\log_k N)$
	Topology-aware hierarchical tree	LL only	$O(MN)$	$O(\log_k N_{\text{remote}} + \log_k N_{\text{intra}})$
AlltoAll	P2P / general all-push	No	$O(MN^2)$	$O(1)$
FCollect (AllGather)	NVLS one-shot	LL only	$O(MN^2)$	$O(1)$
	Generic all-push	Both	$O(MN^2)$	$O(1)$
Reduce (AllReduce)	NVLS one-shot	No	$O(MN^2)$	$O(1)$
	NVLS two-shot	No	$O(MN)$	$O(1)$
	k -ary recursive exchange	No	$O(MN \times k \log_k N)$	$O(\log_k N)$
	Hierarchical Icollect	Inherited from FCollect	$O(MN^2)$	$O(1)$
	(Segmented) linear AllReduce	No	$O(MN^2)$	Direct LD/ST: $O(1)$; Segmented: $O(SN)$
ReduceScatter	NVLS one-shot	No	$O(MN^2)$	$O(1)$
	Generic all-push	No	$O(MN^2)$	$O(1)$

AND $S = \lceil M/B_{\text{seg}} \rceil$ DENOTES THE NUMBER OF SCRATCH-LIMITED SEGMENTS IN SEGMENTED ALLREDUCE.

Collective Algorithm Example: Recursive Exchange for Allreduce

Collective Algorithm Example: Recursive Exchange for Allreduce

Step 1: Group
& Reduce
(k=2)



$\oplus$ the reduce op

$\oplus i..j$ stands for $i \oplus i + 1 \oplus \dots \oplus j$

Collective Algorithm Example: Recursive Exchange for Allreduce

Step 1: Group
& Reduce
(k=2)



Step 2:
Recursive
Exchange (k=2,
Butterfly/
Hypercube)

Phase 1
(stride
1)



⊕ the reduce op
⊕i..j stands for $i \oplus i + 1 \oplus \dots \oplus j$

Collective Algorithm Example: Recursive Exchange for Allreduce

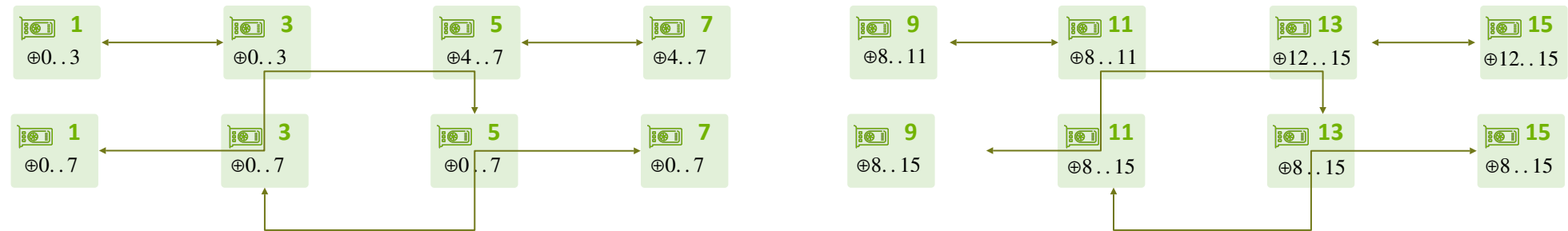
Step 1: Group
& Reduce
(k=2)



Step 2:
Recursive
Exchange (k=2,
Butterfly/
Hypercube)

Phase 1
(stride
1)

Phase 2
(stride
2)



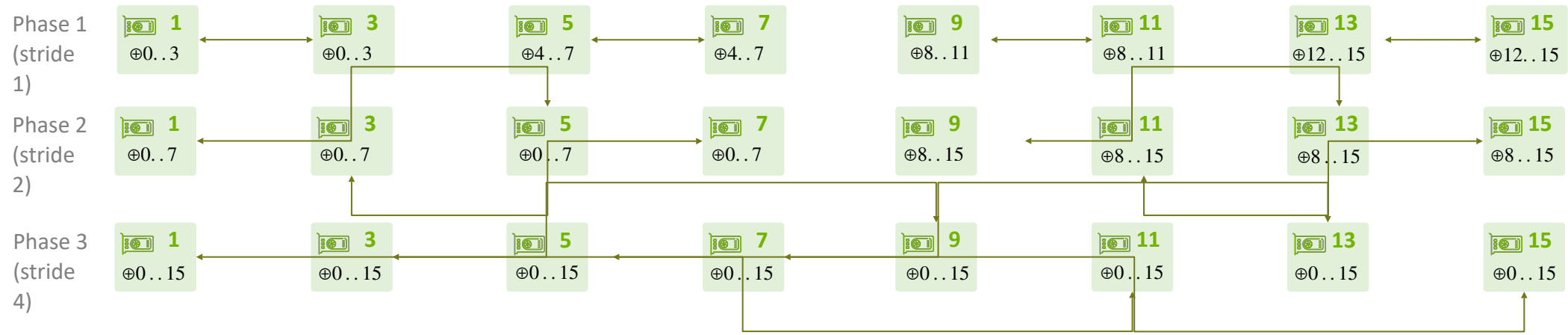
⊕ the reduce op
⊕i..j stands for $i \oplus i + 1 \oplus \dots \oplus j$

Collective Algorithm Example: Recursive Exchange for Allreduce

Step 1: Group
& Reduce
(k=2)



Step 2:
Recursive
Exchange (k=2,
Butterfly/
Hypercube)



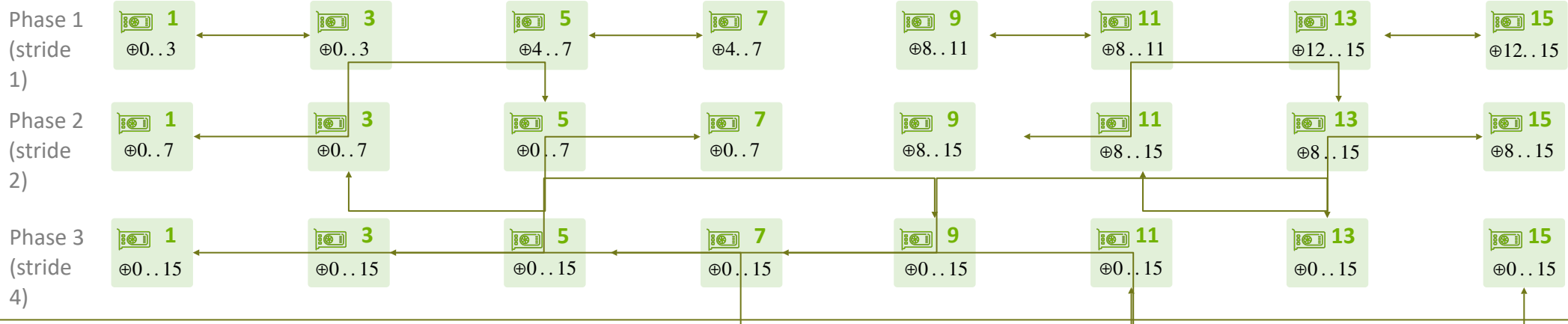
⊕ the reduce op
⊕i..j stands for $i \oplus i + 1 \oplus \dots \oplus j$

Collective Algorithm Example: Recursive Exchange for Allreduce

Step 1: Group
& Reduce
(k=2)



Step 2:
Recursive
Exchange (k=2,
Butterfly/
Hypercube)



Step 3:
Broadcast
(k=2)



$\oplus$ the reduce op

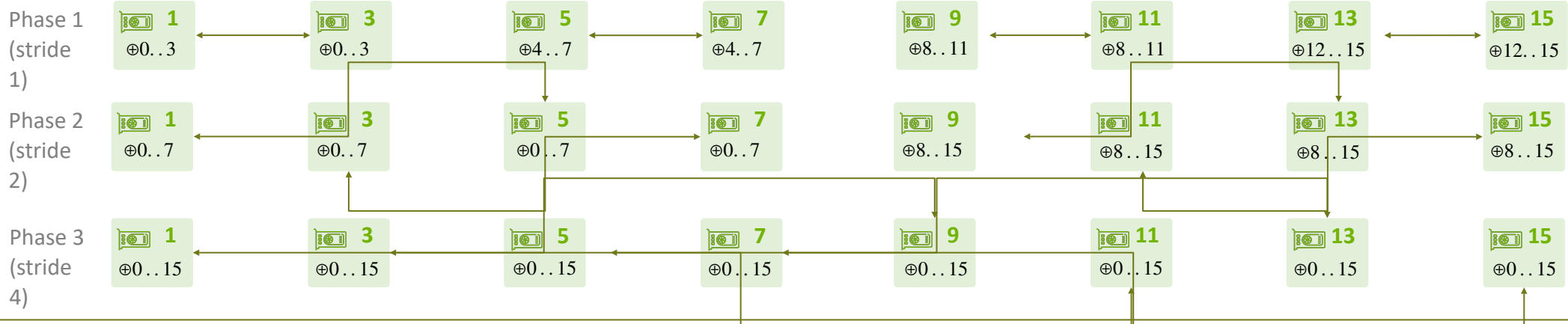
$\oplus i..j$ stands for $i \oplus i + 1 \oplus \dots \oplus j$

Collective Algorithm Example: Recursive Exchange for Allreduce

Step 1: Group
& Reduce
(k=2)



Step 2:
Recursive
Exchange (k=2,
Butterfly/
Hypercube)



Step 3:
Broadcast
(k=2)



⊕ the reduce op

⊕i..j stands for $i \oplus i + 1 \oplus \dots \oplus j$

- Logarithmic latency even in the absence of NVLS

NVSHMEM Device-side Collectives Run on a Single CTA

NVSHMEM Device-side Collectives Run on a Single CTA

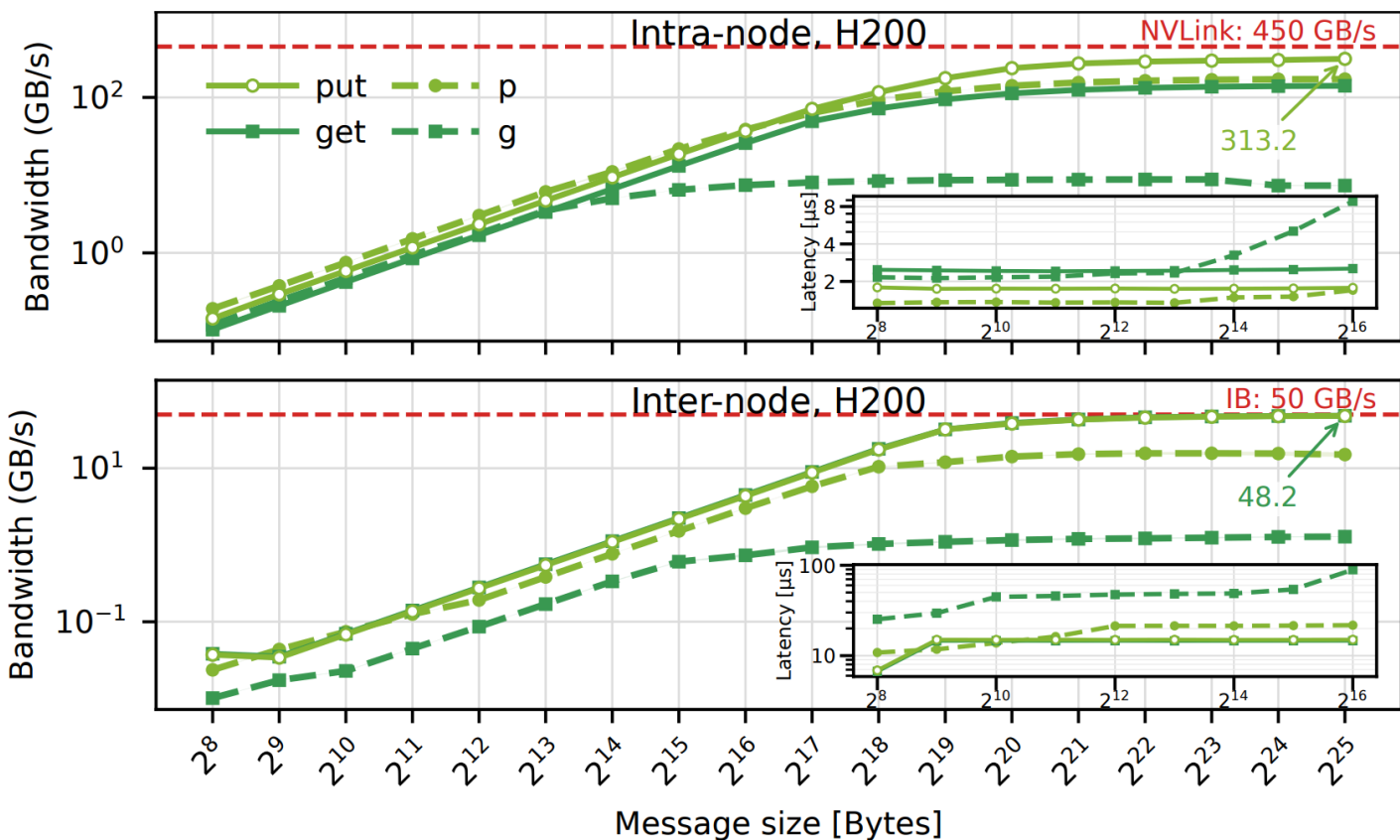
- Alongside `_grid`, `_warp`, `_block` API versions, the absence of public `_grid` variants is deliberate
 - Cross-CTA synchronization inside a running kernel is expensive
 - Better end this kernel and launch a host-side `_on_stream` communication kernel instead
 - It is also consistent with the team model: one team owns one pSync region

One-sided RMA on H200

- 8 x H200 SXM5 per node
- 150 GB/s intra-node NVLink
- 50 GB/s CX7 IB per GPU

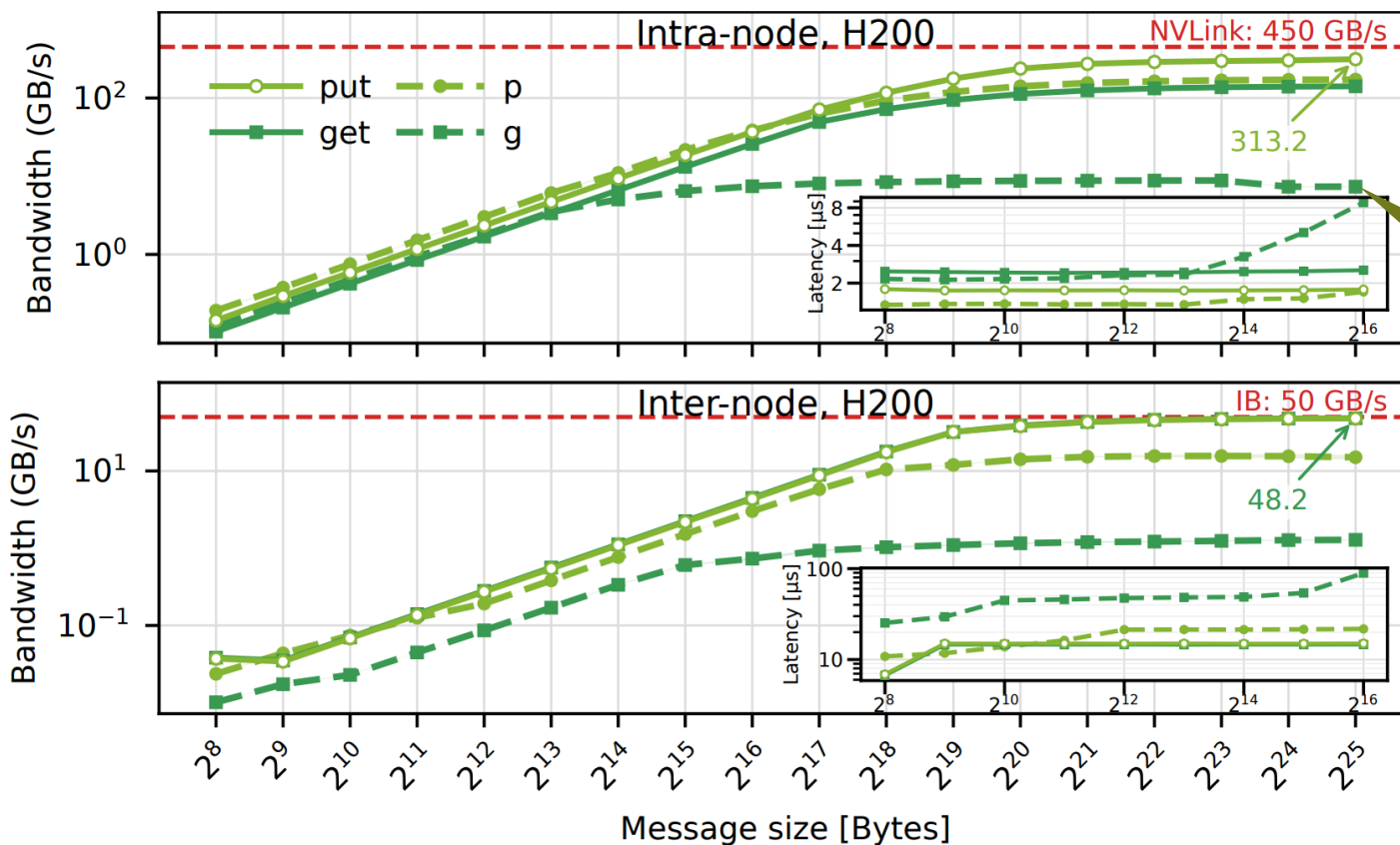
One-sided RMA on H200

- 8 x H200 SXM5 per node
- 150 GB/s intra-node NVLink
- 50 GB/s CX7 IB per GPU



One-sided RMA on H200

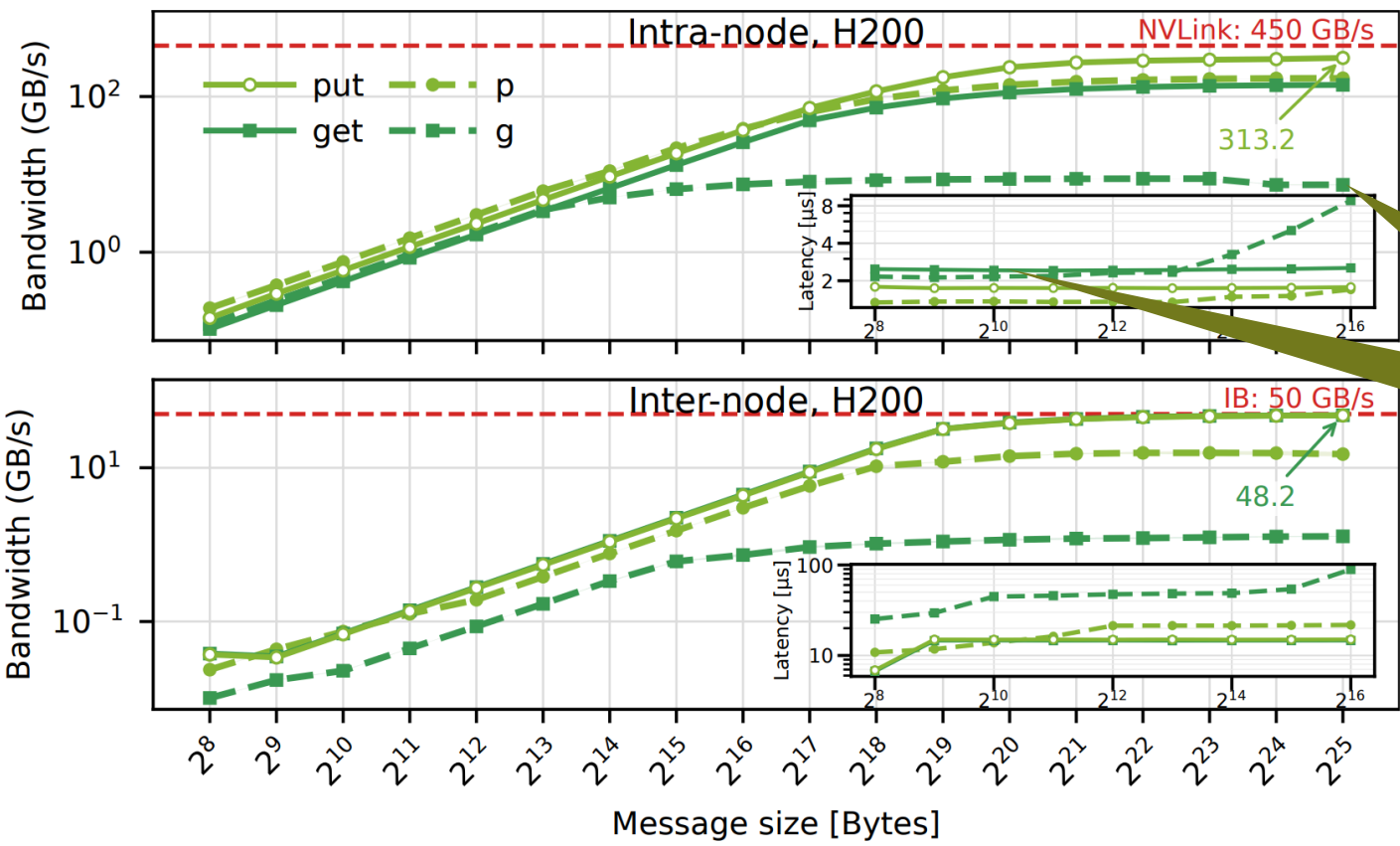
- 8 x H200 SXM5 per node
- 150 GB/s intra-node NVLink
- 50 GB/s CX7 IB per GPU



Intranode scalar g:
< 9 GB/s

One-sided RMA on H200

- 8 x H200 SXM5 per node
- 150 GB/s intra-node NVLink
- 50 GB/s CX7 IB per GPU

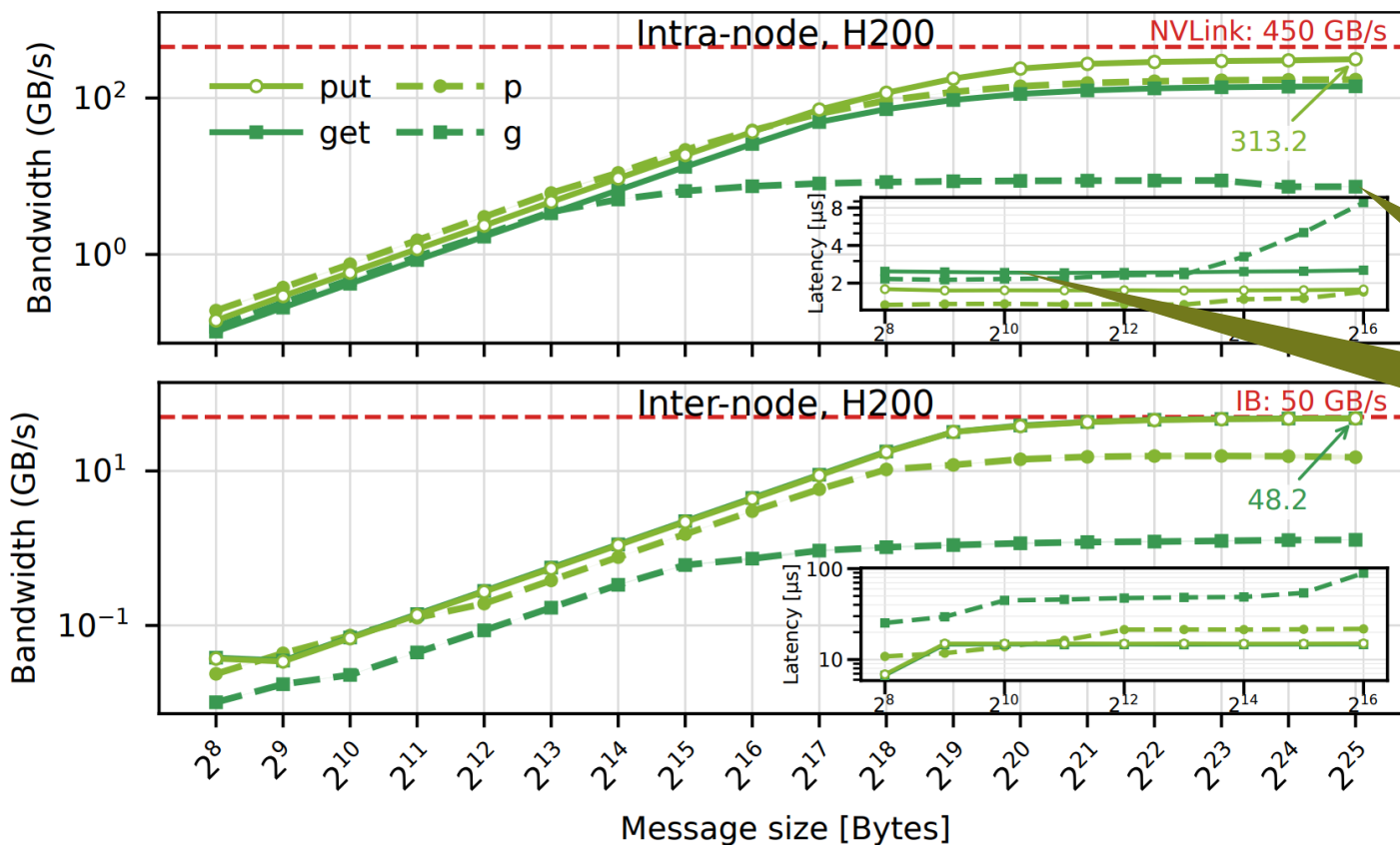


Intranode scalar g:
< 9 GB/s

g, p 10~25%
better latency
for small messages

One-sided RMA on H200

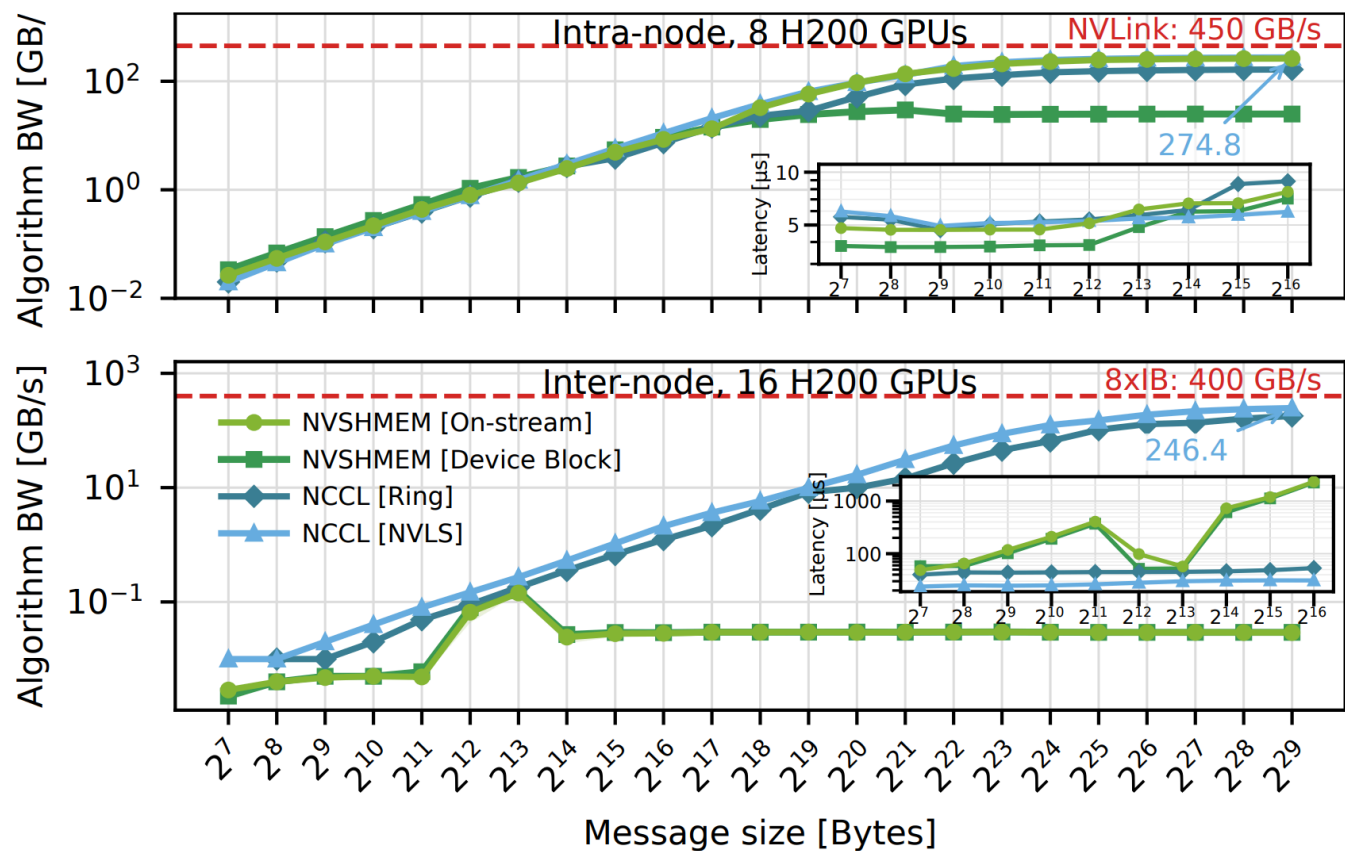
- 8 x H200 SXM5 per node
- 150 GB/s intra-node NVLink
- 50 GB/s CX7 IB per GPU



→ Treat NVSHMEM scalar operations as control primitives. Prioritize push-based operations.

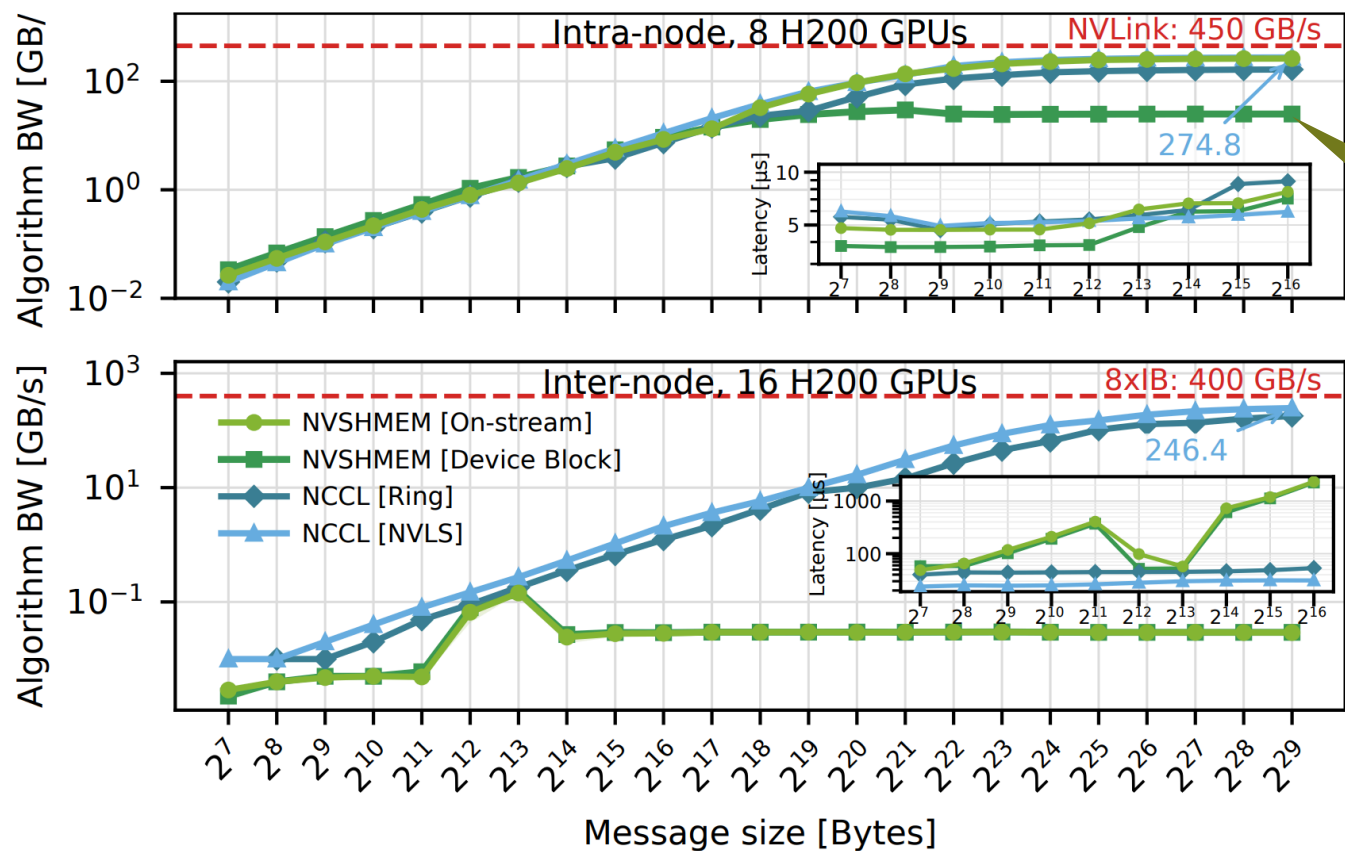
AllReduce: NVSHMEM vs NCCL on H200

- 8 x H200 SXM5 per node
- 150 GB/s intra-node NVLink
- 50 GB/s CX7 IB per GPU



AllReduce: NVSHMEM vs NCCL on H200

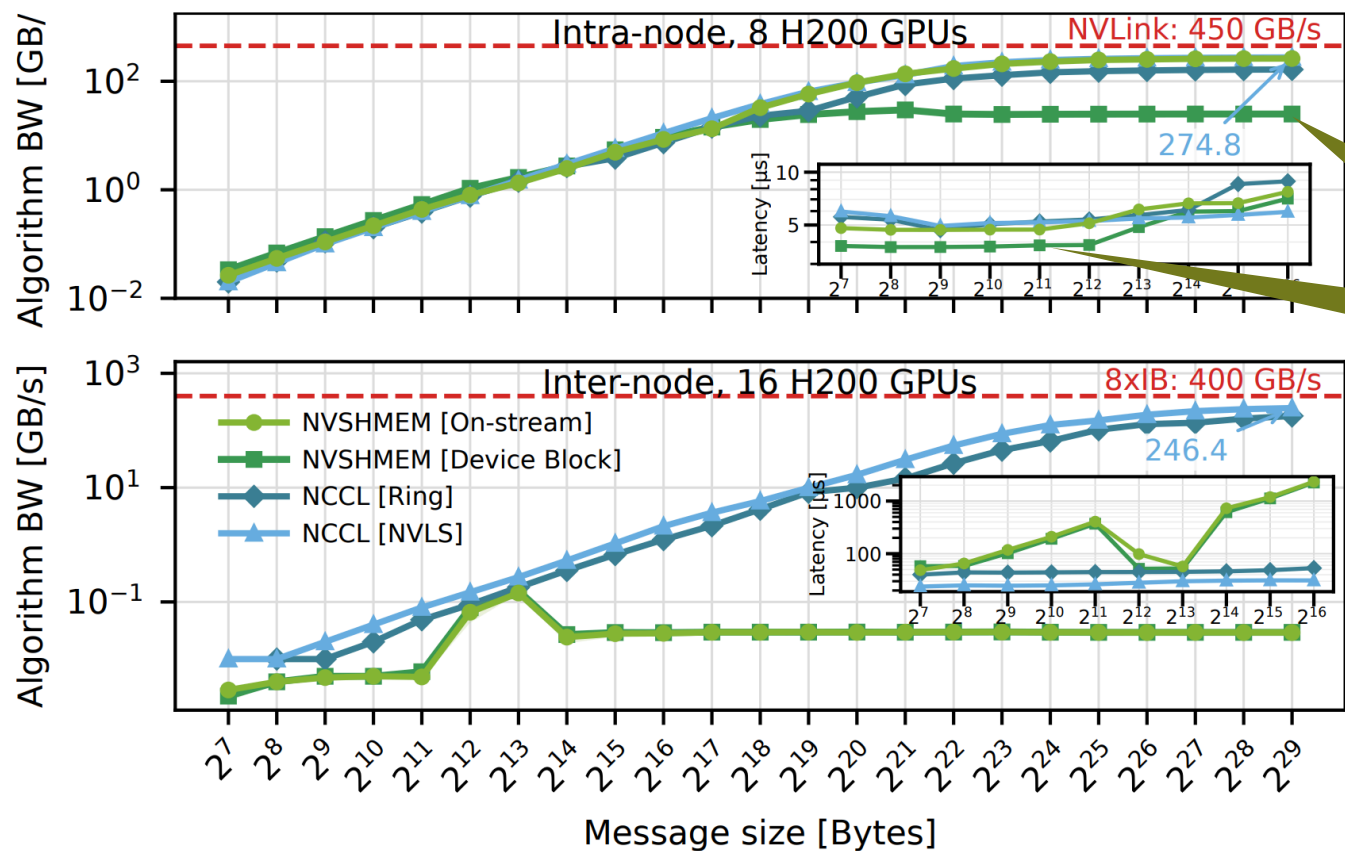
- 8 x H200 SXM5 per node
- 150 GB/s intra-node NVLink
- 50 GB/s CX7 IB per GPU



NVSHMEM device-side path: 30 GB/s

AllReduce: NVSHMEM vs NCCL on H200

- 8 x H200 SXM5 per node
- 150 GB/s intra-node NVLink
- 50 GB/s CX7 IB per GPU

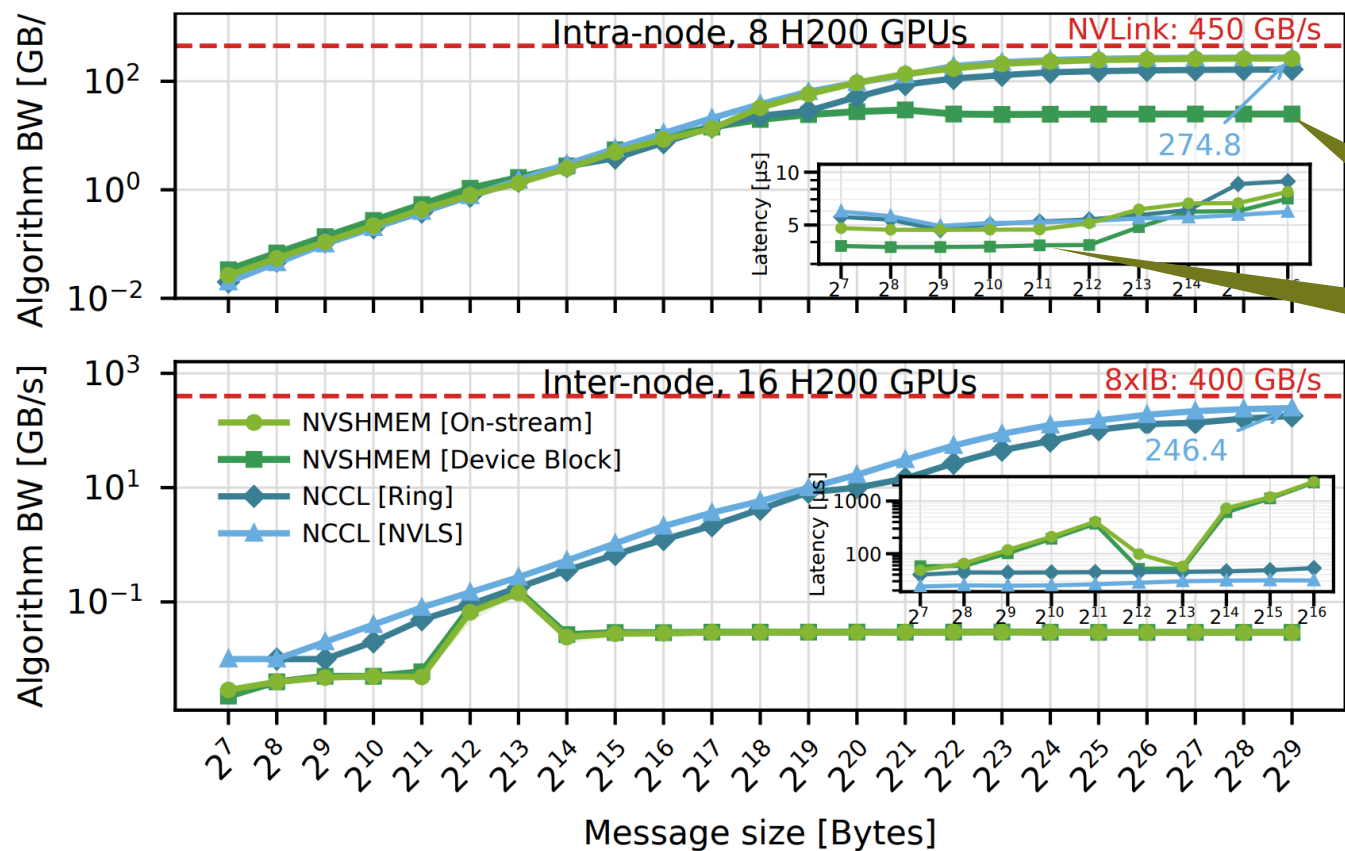


NVSHMEM device-side path: 30 GB/s

but lower latency for small messages

AllReduce: NVSHMEM vs NCCL on H200

- 8 x H200 SXM5 per node
- 150 GB/s intra-node NVLink
- 50 GB/s CX7 IB per GPU



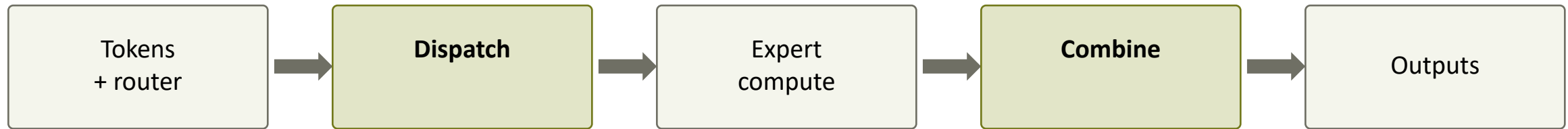
NVSHMEM device-side path: 30 GB/s

but lower latency for small messages

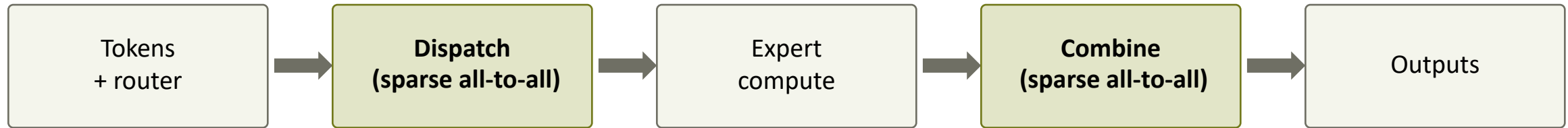
→ Treat NVSHMEM collective operations as control primitives as well.

Example: DeepEP (MoE Communication Library)

Example: DeepEP (MoE Communication Library)

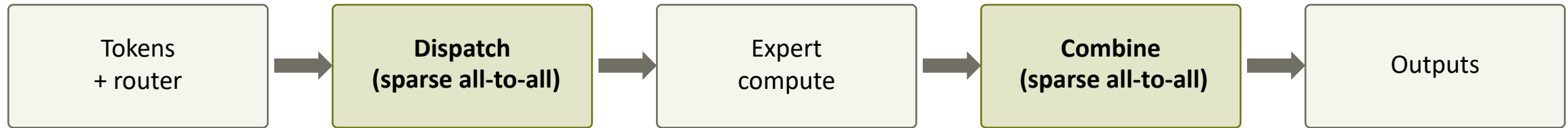


Example: DeepEP (MoE Communication Library)



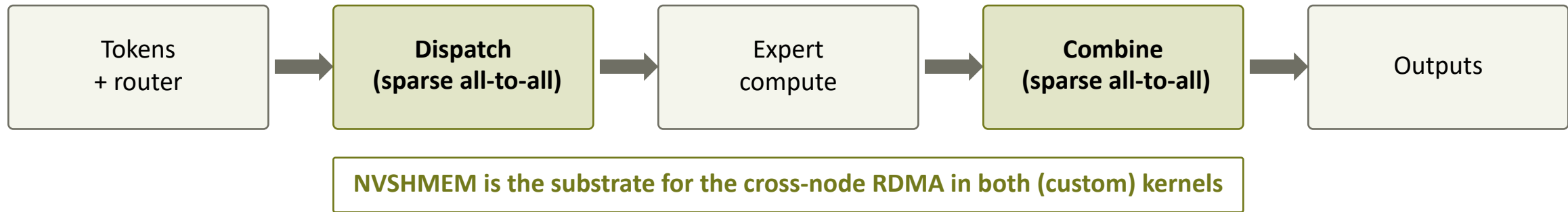
Example: DeepEP (MoE Communication Library)

- In EP, the volume between any pair of ranks is **data-dependent**
 - Hard to express efficiently with off-the-shelf collective libraries



Example: DeepEP (MoE Communication Library)

- In EP, the volume between any pair of ranks is **data-dependent**
 - Hard to express efficiently with off-the-shelf collective libraries



vs NCCL Device API and GIN: Converging Designs

vs NCCL Device API and GIN: Converging Designs

- NCCL added device-initiated operations and symmetric-memory support through **NCCL device API**
 - but target different abstractions



Memory view

vs NCCL Device API and GIN: Converging Designs

- NCCL added device-initiated operations and symmetric-memory support through **NCCL device API**
 - but target different abstractions

	NVSHMEM
Memory view	Flat remote-memory view over all PEs

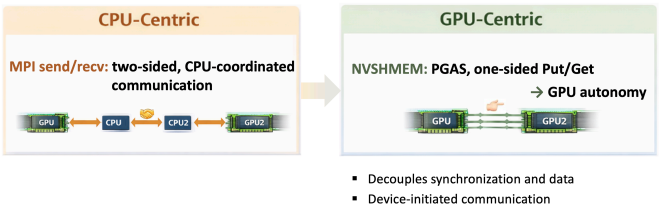
vs NCCL Device API and GIN: Converging Designs

- NCCL added device-initiated operations and symmetric-memory support through **NCCL device API**
 - but target different abstractions

	NVSHMEM	NCCL Device API
Memory view	Flat remote-memory view over all PEs	Explicit scale-up vs scale-out domains

Conclusions

Background: From CPU-Centric to GPU-Centric Communication



More of SPCL's research:

 youtube.com/@spcl  **210+ Talks**

 twitter.com/spcl_eth  **1.6K+ Followers**

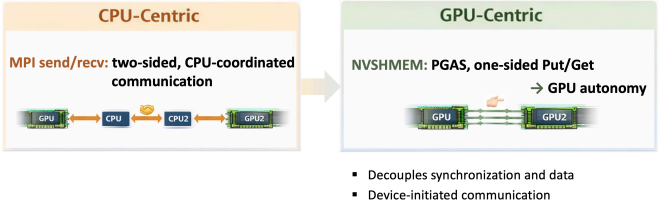
 github.com/spcl  **5.6K+ Stars**

... or spcl.ethz.ch



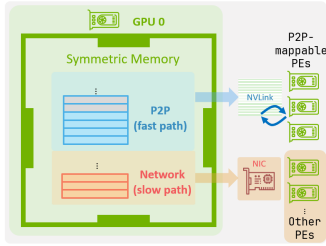
Conclusions

Background: From CPU-Centric to GPU-Centric Communication



Symmetric Memory and Remote Memory Access

- Symmetric object addressed by `<symmetric_address, destination_PE>`
 - If destination PE is **P2P direct-mapped**: **fast path** via the map transport, e.g. NVLink
 - Otherwise: **slow path via network**, e.g. InfiniBand
 - Everything is GPU-initiated



More of SPCL's research:

 youtube.com/@spcl **210+ Talks**

 twitter.com/spcl_eth **1.6K+ Followers**

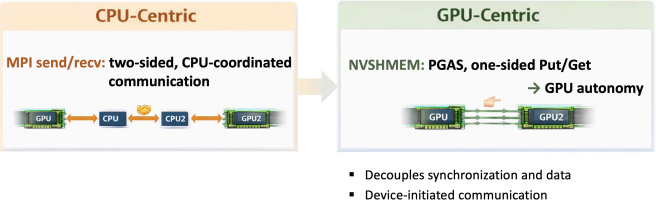
 github.com/spcl **5.6K+ Stars**

... or spcl.ethz.ch



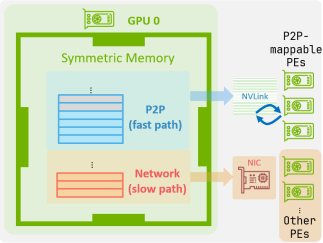
Conclusions

Background: From CPU-Centric to GPU-Centric Communication



Symmetric Memory and Remote Memory Access

- Symmetric object addressed by `<symmetric_address, destination_PE>`
 - If destination PE is **P2P direct-mapped**: **fast path** via the map transport, e.g. NVLink
 - Otherwise: **slow path** via network, e.g. InfiniBand
 - Everything is GPU-initiated



More of SPCL's research:

 youtube.com/@spcl

210+ Talks

 twitter.com/spcl_eth

1.6K+ Followers

 github.com/spcl

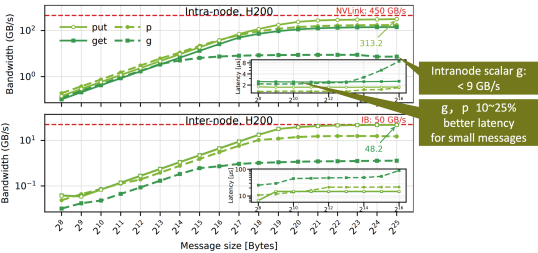
5.6K+ Stars

... or spcl.ethz.ch



One-sided RMA on H200

- 8 x H200 5XMS per node
- 150 GB/s intra-node NVLink
- 50 GB/s CX7 IB per GPU



→ Treat NVSHMEM scalar operations as control primitives. Prioritize push-based operations.

Conclusions

More of SPCL's research:



youtube.com/@spcl

210+ Talks



twitter.com/spcl_eth

1.6K+ Followers



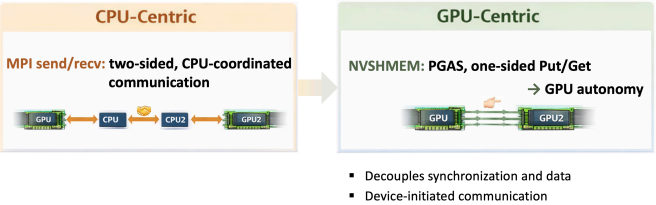
github.com/spcl

5.6K+ Stars

... or spcl.ethz.ch

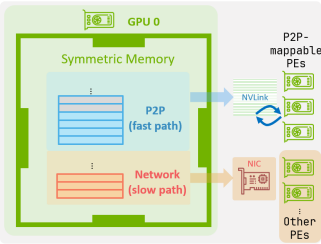


Background: From CPU-Centric to GPU-Centric Communication



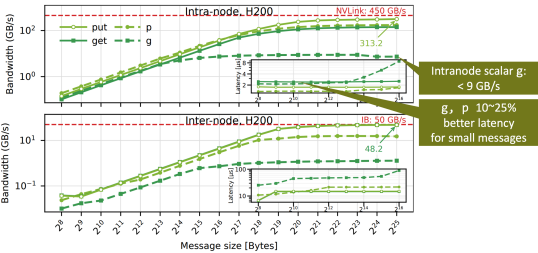
Symmetric Memory and Remote Memory Access

- Symmetric object addressed by `<symmetric_address, destination_PE>`
 - If destination PE is **P2P direct-mapped**: **fast path** via the map transport, e.g. NVLink
 - Otherwise: **slow path** via network, e.g. InfiniBand
 - Everything is GPU-initiated



One-sided RMA on H200

- 8 x H200 5XMS per node
- 150 GB/s intra-node NVLink
- 50 GB/s CX7 IB per GPU



→ Treat NVSHMEM scalar operations as control primitives. Prioritize push-based operations.

Example: DeepEP (MoE Communication Library)

- In EP, the volume between any pair of ranks is **data-dependent**
 - Hard to express efficiently with off-the-shelf collective libraries

